
Quantitative Big Imaging - Building and Augmenting Datasets

Anders Kaestner

Feb 26, 2026

CONTENTS

0.1	Ground Truth: Building and Augmenting Datasets	1
0.2	Famous Datasets	6
0.3	Purpose of different types of Datasets	12
0.4	Building your own data sets	20
0.5	Dataset Problems	22
0.6	Augmentation	26
0.7	Baselines	31
0.8	Data frames - managing feature tables	43
0.9	Summary	54

This is the lecture notes for the 2nd lecture of the Quantitative big imaging class given during the spring semester 2022 at ETH Zurich, Switzerland.

0.1 Ground Truth: Building and Augmenting Datasets

Quantitative Big Imaging ETHZ: 227-0966-00L

0.1.1 Today's lecture

Creating Datasets

- Famous Data sets
- Types of Datasets
- What makes a good data set?
- Building your own
 - “scrape, mine, move, annotate, review, and preprocess” - Kathy Scott
 - Tools to use
 - Simulations

0.1.2 Today's lecture (continued)

Augmentation

- How can you artificially increase the size of your dataset?
- What are the limits of these increases

Baselines

- What is a baseline?
- Example: Nearest Neighbor

0.1.3 Let's load some modules for the notebook

```
import matplotlib.pyplot as plt
import matplotlib as mpl
import numpy as np
import skimage as ski
import skimage.io as io
from skimage.morphology import disk
import scipy.ndimage as ndimage
from tensorflow.keras.datasets import mnist
import tensorflow_datasets as tfds
from skimage.util import montage as montage2d

%matplotlib inline
mpl.rcParams['figure.dpi'] = 100
```

0.1.4 References

- Revisiting **Unreasonable Effectiveness of Data** in Deep Learning Era
- Data science ... without any data
- Building Datasets
 - Python Machine Learning 2nd Edition by Sebastian Raschka, Packt Publishing Ltd. 2017
 - Chapter 2: Building Good Datasets:
 - A Standardised Approach for Preparing Imaging Data for Machine Learning Tasks in Radiology
- Creating Datasets / Crowdsourcing
 - Mindcontrol: A web application for brain segmentation quality control
 - Combining citizen science and deep learning to amplify expertise in neuroimaging
- Augmentation tools
 - ImgAug
 - Augmentor

0.1.5 Motivation

Why other peoples data?

Most of you taking this class are rightfully excited to learn about new tools and algorithms to analyzing *your* data.

This lecture is a bit of an anomaly and perhaps disappointment because it doesn't cover any algorithms, or tools.

- You might ask, why are we spending so much time on datasets?
- You already collected data (sometimes lots of it) that is why you took this class?!

... let's see what some other people say

Sean Taylor (Research Scientist at Facebook)

This tweet tells us that you shouldn't put too much belief in AI without providing carefully prepared data set. Machine learning methods perform only so good as the data it was trained with. You need a data set that covers all extremes of the phenomena that you want to model.

Andrej Karpathy (Director of AI at Tesla)

This slide by Andrej Karpathy shows the importance of correct data set in a machine learning project. Typically, you should spend much more time on collecting representative data for your models than building the models. Unfortunately, this is not the case for many PhD projects where the data usually is scarce. Much for the reason that it is really hard to come by the data. You may only have a few beam slots allocated for your experiments and this is the data you have to live with.



Sean J. Taylor
@seanjtaylor

I'm as enthusiastic about the future of AI as (almost) anyone, but I would estimate I've created 1000X more value from careful manual analysis of a few high quality data sets than I have from all the fancy ML models I've trained combined.

1:33 am · 20 Feb 2018 · Twitter Web Client

392 Retweets 1.3K Likes

Fig. 1: Realistic thoughts about AI.

Why you need to improve your training data, and how to do it

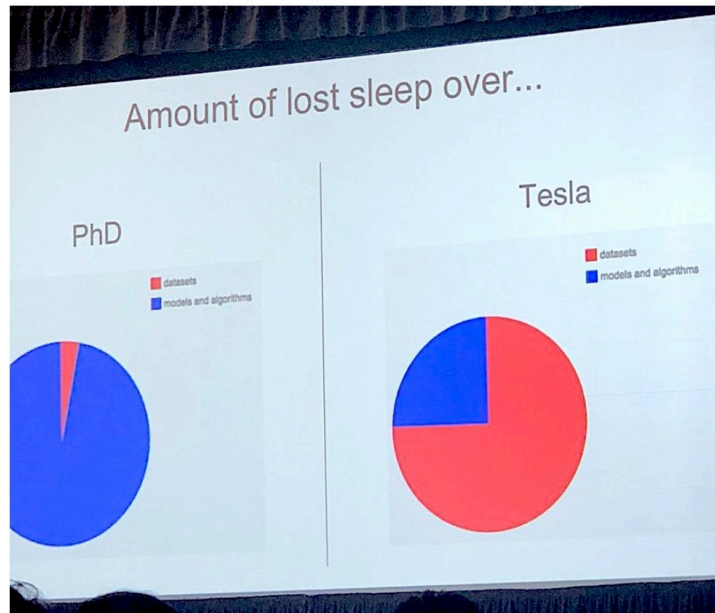


Photo by Lisha Li

Andrej Karpathy showed this slide as part of his talk at [Train AI](#) and I loved it! It captures the difference between deep learning

Fig. 2: Time spent on different tasks.

Kathy Scott (Image Analytics Lead at Planet Labs)

Yet another tweet that implies that many data scientist actually spend more time on preparing the data than developing new models. The training is less labor demanding, the computer is doing that part of the job.

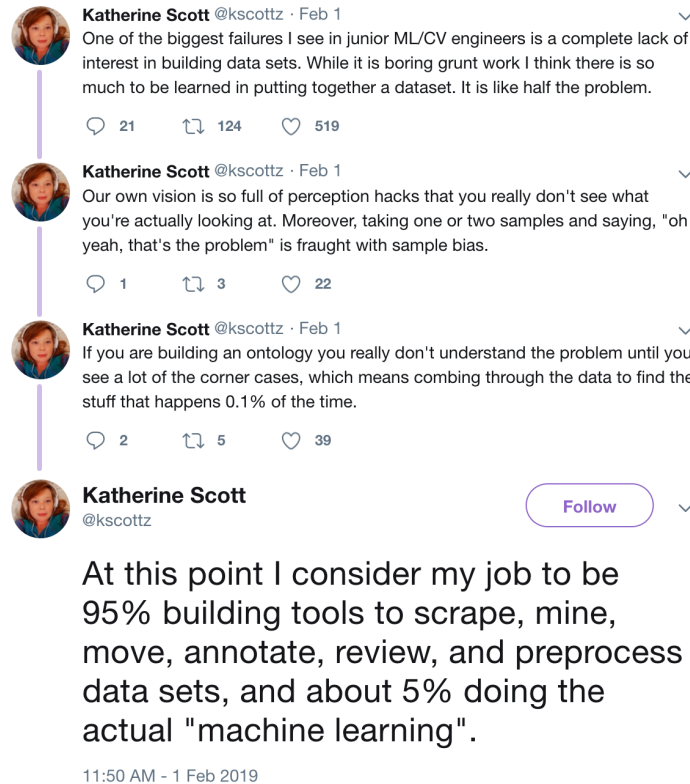


Fig. 3: The importance to spend sufficient time on data preparation.

0.1.6 Data is important

It probably isn't the *new oil*, but it forms an essential component for building modern tools today.

Testing good algorithms *requires* good data

- If you don't know what to expect – *how do you know your algorithm worked?*
- If you have dozens of edge cases – *how can you make sure it works on each one?*
- If a new algorithm is developed every few hours – *how can you be confident they actually work better?*

Facebook's site has a new version multiple times per day and their app every other day

For machine learning, even building algorithms requires good data

- If you count cells maybe you can write your own algorithm,
- but if you are trying to detect
 - *subtle changes* in cell structure
 - that indicate canceryou probably can't write a list of simple mathematical rules yourself.

0.1.7 Data is reusable

- Well organized and structured data is very easy to reuse.
- Another project can easily combine your data with their data in order to get even better results.

Algorithms are often only prototypes

- messy,
- complicated,
- poorly written,

... especially so if written by students trying to graduate on time.

Data recycling saves time and improves performance

0.1.8 What I ask for in a data set

Starting the explorative analysis of the data to learn what to expect in the analysis. I often ask for examples in the categories

- The Good - These are clean examples of images showing obvious features in noise and artifact free conditions. These are good for the initial steps building an analysis workflow.
- The Bad - These are the typical images you expect to obtain in your experiments. They form the foundation for your analysis.
- The Ugly - These are the corner cases that may appear in your experiments. They are noisy, low contrast, artifact burdened and maybe even unevenly illuminated. The expected information is often hard to come by in these images. Thus, they represent the worst case scenario for your analysis.



Your analysis workflow should at least be able to handle the Bad as this is the core of your experiment data. With the ability to handle the Ugly you introduce robustness in your analysis.

0.2 Famous Datasets

The primary success of datasets has been shown through the most famous datasets collected.

Here I show

- Two of the most famous general datasets
 - MNIST Digits
 - ImageNET
- and one of the most famous medical datasets.
 - BRATS

The famous datasets are important for basic network training.

0.2.1 MNIST Digits

Modified NIST (National Institute of Standards and Technology) created a list of handwritten digits.

This list is a popular starting point for many machine learning projects. The images are already labeled and are also nicely prepared to about the same size and also very high SNR. These properties makes it a great toy data set for first testing.



Fig. 1: A selection of hand written numbers from the MNIST data base

0.2.2 ImageNet

- ImageNet is an image database
 - organized according to the WordNet hierarchy (currently only the nouns),
 - each node of the hierarchy is depicted by hundreds and thousands of images.
- 1000 different categories and >1M images.
- Not just dog/cat, but wolf vs german shepard,

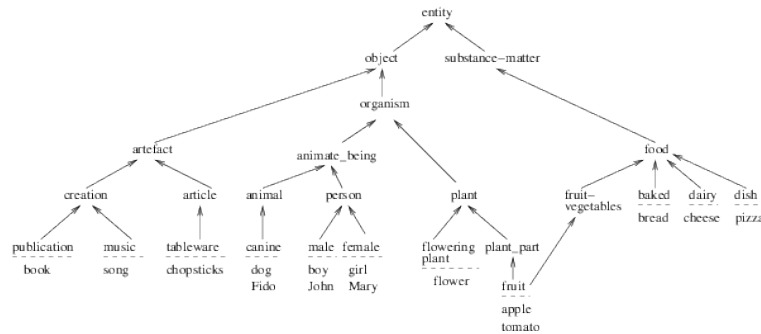


Fig. 2: Hierarchical structure of the WordNet database.

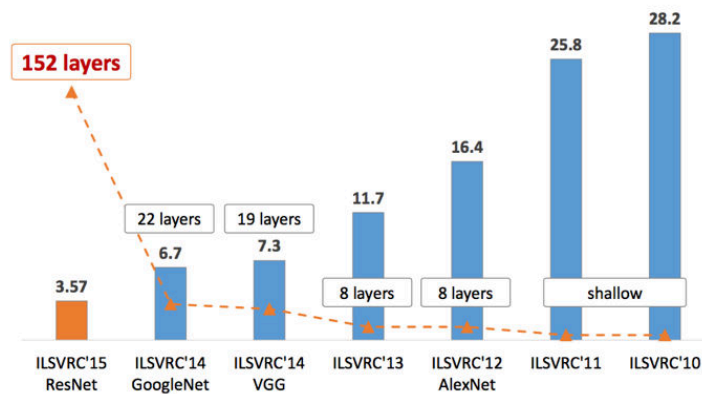


Fig. 3: Error rates for different classifiers on the same data set.

CNN architectures

0.2.3 What story did these datasets tell?

Each of these datasets is very different from images with fewer than 1000 pixels to images with more than 100MPx, but what they have in common is how their analysis has changed.

All of these datasets used to be analyzed by domain experts with hand-crafted features.

- A handwriting expert using graph topology to assign images to digits
- A computer vision expert using gradients common in faces to identify people in ImageNet
- A biomedical engineer using knowledge of different modalities to fuse them together and cluster healthy and tumorous tissue



Fig. 4: Domain experts use their experience to analyze data

Starting in the early 2010s, the approaches of deep learning began to improve and become more computationally efficient. With these techniques groups with **absolutely no domain knowledge** could begin building algorithms and winning contests based on these datasets.



Fig. 5: Data scientist don't have domain specific knowledge, they use available data for the analysis.

All of these datasets used to be analyzed by **domain experts** with hand-crafted features.

- A handwriting expert using graph topology
- A computer vision expert to identify people in ImageNet
- A biomedical engineer cluster healthy and tumorous tissue
- the approaches of deep learning began to improve and become more computationally efficient.
- groups with **absolutely no domain knowledge** could winning contests based on datasets

0.2.4 So Deep Learning always wins?

No, that isn't the point of this lecture.

Even if you aren't using deep learning the point of these stories is having

- well-labeled,
- structured,
- and organized datasets

makes your problem *a lot more accessible* for other groups and enables a variety of different approaches to be tried.

Ultimately it enables better solutions to be made and you to be confident that the solutions are in fact better.

How to work with someone else's data

Inherited datasets are like inherited toothbrushes: using them is an act of desperation.

Collecting your own data is a luxury not everyone can afford.

Inherited data are easier to get but harder to trust.

C. Kozyrkov, 2020

0.2.5 Data set ethics

Core Ethical Principles

- Transparency & Reproducibility
- Privacy & Confidentiality
- Fairness & Bias Mitigation
- Integrity & Avoidance of Data Manipulation
- Responsible Use & Governance

Best Practices for Ethical Data Management

- Obtain Informed Consent

Ensure participants understand data collection, usage, and storage implications.

- Anonymize Data Where Possible

Remove personally identifiable information (PII) before sharing.

- Use Ethical Review Committees

Seek approvals for research involving human data.

- Ensure Data Security

Use encryption and controlled access to sensitive datasets.

- Promote Open & FAIR Data

Share data responsibly under proper licensing.

- Audit for Bias & Fairness

Regularly assess datasets for potential ethical concerns.

- Document Everything

Maintain transparency in data provenance, transformations, and analytical methods.

The FAIR principle

Open data is a central requirement these days.

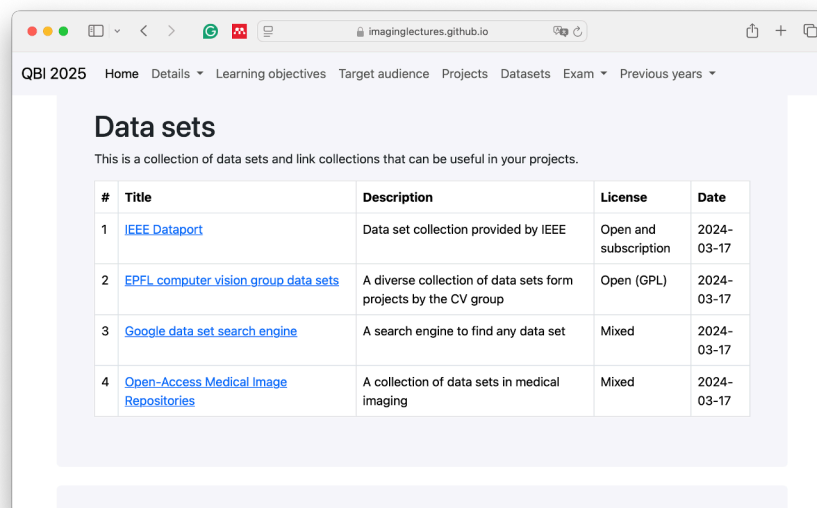
- **F** indable
- **A** ccessible
- **I** nteroperable
- **R** eusable

Wilkinson et al. 2016

PaNOSC

The Photon and Neutron Open Science Cloud (PaNOSC)

0.2.6 Data set sources



#	Title	Description	License	Date
1	IEEE Dataport	Data set collection provided by IEEE	Open and subscription	2024-03-17
2	EPFL computer vision group data sets	A diverse collection of data sets form projects by the CV group	Open (GPL)	2024-03-17
3	Google data set search engine	A search engine to find any data set	Mixed	2024-03-17
4	Open-Access Medical Image Repositories	A collection of data sets in medical imaging	Mixed	2024-03-17

Fig. 6: Some dataset databases listed on the course web page.

Medical imaging: BRATS

Segmenting Tumors in Multimodal MRI Brain Images.

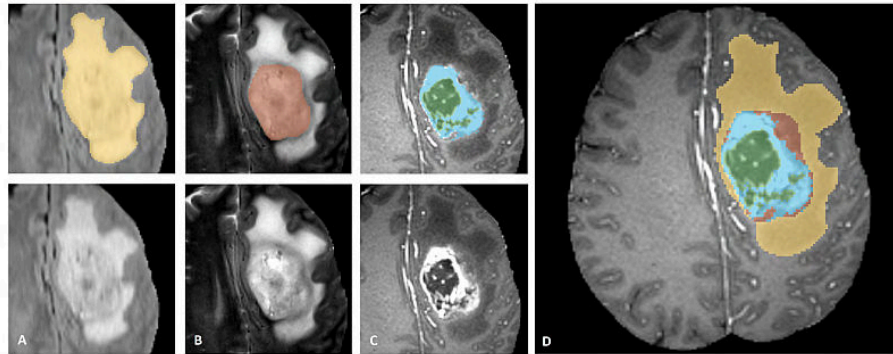


Fig. 7: Images of brain tumors from the BRATS database.

Other Datasets

- [Grand-Challenge.org](#) a large number of challenges in the biomedical area
- [Kaggle Datasets](#)
- [Google Dataset Search](#)
- [Wikipedia](#) provides a comprehensive list categorized into different topics

0.2.7 What makes a good dataset?

A good data set is characterized by

This means that...

Lots of images

Particularly if you have:

- Complicated problems
- Very subtle differences
 - a lung tumor looks mostly like normal lung tissue, but it is in a place it shouldn't be
- Class imbalance

Small datasets can be useful, but here the bigger the better

Lots of diversity

- Is it what data ‘in the wild’ really looks like?
- Lots of different
 - Scanners/reconstruction algorithms,
 - noise levels,
 - illumination types,
 - rotation,
 - colors, ...
- Many examples from different categories

If you only have one male with breast cancer,
it will be hard to generalize exactly what that looks like

Meaningful labels

For labels to be meaningful they should fulfill the following criteria:

- Clear task or question
- Unambiguous
- Non-obvious
- Able to be derived from the image alone
- Quantitative!

The labels must make it possible to give the answer to a clear question. This requires that the label must be the same for any labeler who independently observe the data.

A non-obvious label cannot be calculated from the data. The label is then trivial and is not needed.

A label saying an image is bright is not a helpful label because you could look at the histogram and say that_

A label that someone cannot afford insurance is interesting but it would be nearly impossible to determine that from an X-ray of their lungs_

0.3 Purpose of different types of Datasets

- Classification
- Regression
- Segmentation
- Detection
- Other

0.3.1 Classification

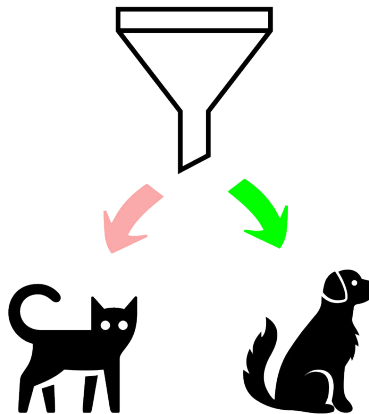


Fig. 1: Classification example with cats and dogs.

- Taking an image and putting it into a category
- Each image should have exactly one category
- The categories should be non-ordered
- Example:
 - Cat vs Dog
 - Cancer vs Healthy

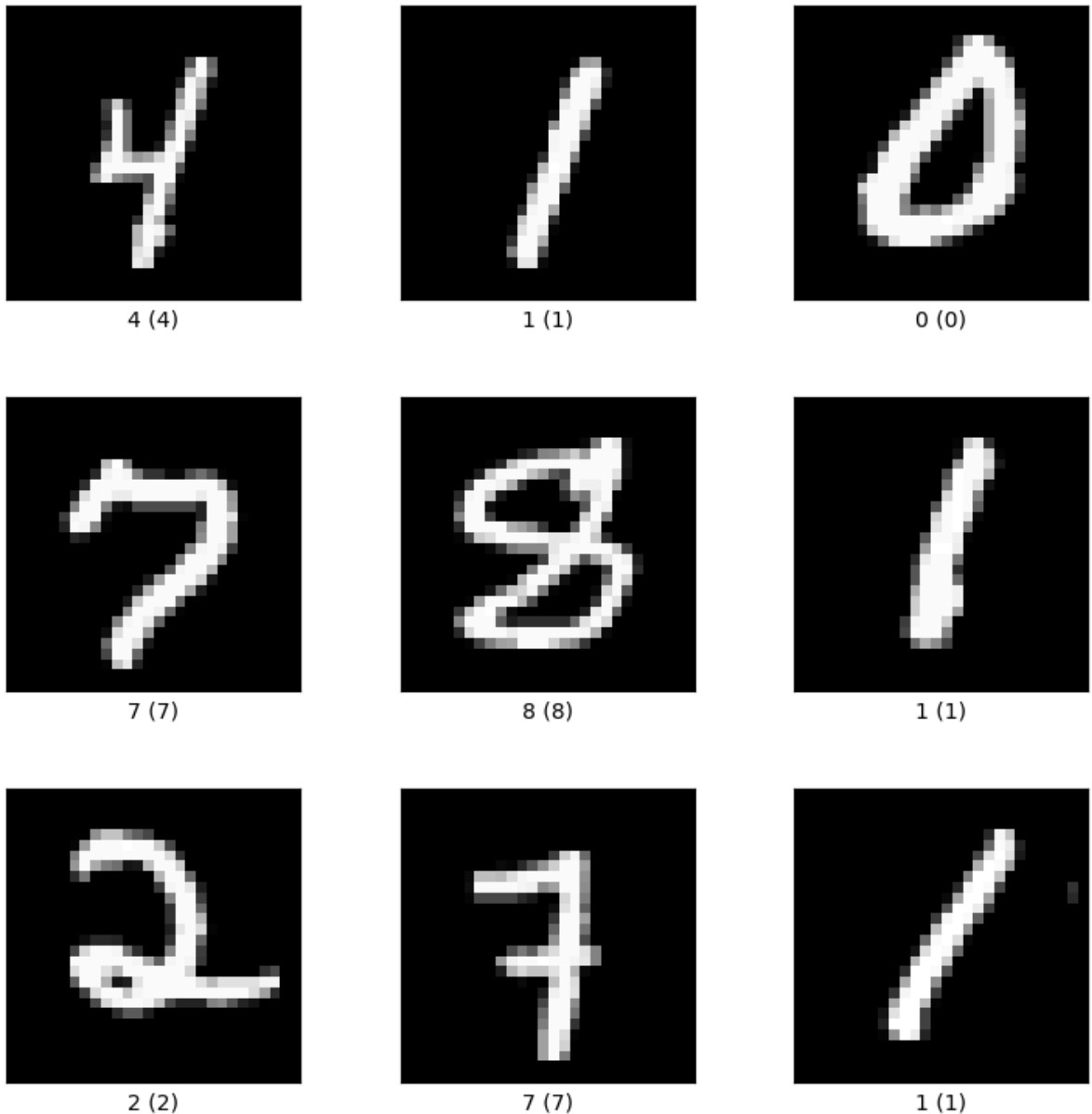
Classification example

In classification you want to observe an image and quickly provide it with a category. Like in the MNIST example which is designed for recognition of handwritten numbers. Each image has a label telling which number it represents.

```
ds, info = tfds.load('mnist', split='train', with_info=True)
```

```
fig = tfds.show_examples(ds, info)
```

```
2026-02-26 11:41:48.042836: I metal_plugin/src/device/metal_device.cc:1154] Metal_
↳device set to: Apple M1 Max
2026-02-26 11:41:48.042858: I metal_plugin/src/device/metal_device.cc:296]_
↳systemMemory: 32.00 GB
2026-02-26 11:41:48.042861: I metal_plugin/src/device/metal_device.cc:313]_
↳maxCacheSize: 12.48 GB
2026-02-26 11:41:48.042876: I tensorflow/core/common_runtime/pluggable_device/_
↳pluggable_device_factory.cc:305] Could not identify NUMA node of platform GPU ID_
↳0, defaulting to 0. Your kernel may not have been built with NUMA support.
2026-02-26 11:41:48.042886: I tensorflow/core/common_runtime/pluggable_device/_
↳pluggable_device_factory.cc:271] Created TensorFlow device (/job:localhost/_
↳replica:0/task:0/device:GPU:0 with 0 MB memory) -> physical PluggableDevice_
↳(device: 0, name: METAL, pci bus id: <undefined>)
2026-02-26 11:41:48.099230: W tensorflow/core/kernels/data/cache_dataset_ops._
↳cc:858] The calling iterator did not fully read the dataset being cached. In_
↳order to avoid unexpected truncation of the dataset, the partially cached_
↳contents of the dataset will be discarded. This can happen if you have an input_
↳pipeline similar to `dataset.cache().take(k).repeat()`. You should use `dataset._
↳take(k).cache().repeat()` instead.
2026-02-26 11:41:48.106314: W tensorflow/core/framework/local_rendezvous.cc:404]_
↳Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```



0.3.2 Regression

Regression almost looks like classification at first sight. You still want to put a number related to the image content. But here it is not strictly bound to the provided categories but rather estimating a value which can be found on the regression line fitted to the data.

Taking an image and predicting one (or more) decimal values

Examples:

- Value of a house from the picture taken by owner
- Risk of hurricane from satellite image

Regression example Age from X-Rays

This dataset contains a collection of X-ray radiographs of hands. The purpose of the data is to estimate the age of a child based on the radiograph. This can be done using a regression model.

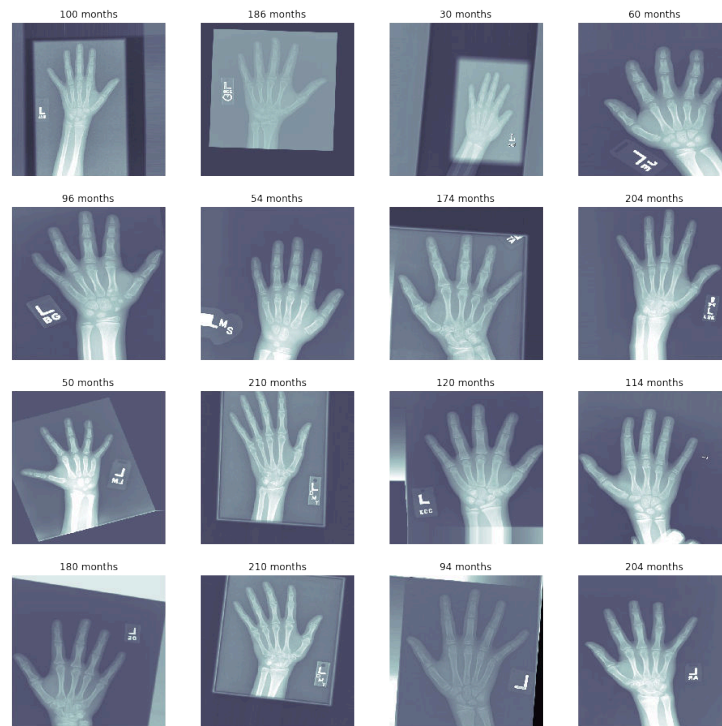


Fig. 2: A collection of X-ray images from children at different ages.

[More details](#)

0.3.3 Segmentation

- Taking an image and predicting one (or more) values for each pixel
- Every pixel needs a label (and a pixel cannot have multiple labels)
- Typically limited to a few (less than 20) different types of objects

Segmentation examples:

- Where a tumor is from an image of the lungs?
- Where streets are from satellite images of a neighborhood?
- Where are the cats and dogs?

Fig. 3: A cats and dogs segmentation example.

Segmentation example: Nuclei in Microscope Images

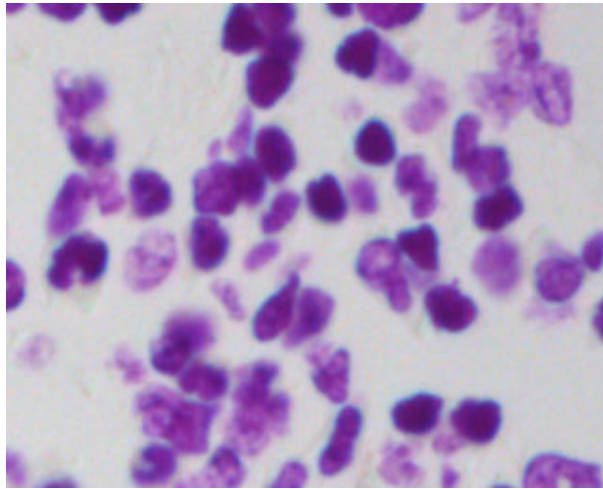


Fig. 4: Sample with cells.



Fig. 5: Labelled cells.

[More details on Kaggle](#)

A segmentation algorithm was able to label all the cells in the original image. A label image has each region set to a unique number. This may be hard to see when the label image is displayed with a gray scale color map. Therefore, it is common to use a color map that clearly shows the individual items. Here, the [Glasbey color map](#) in ImageJ was used.

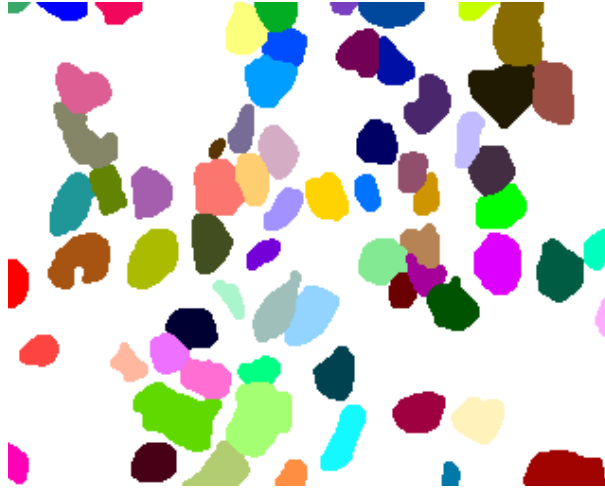


Fig. 6: Labelled cells with colormap.

0.3.4 Detection

Detection is a combination of segmentation and classification in the sense that the location and extents of a feature is determined and is also categorized into some class. The extents don't have to be very precise, it is often a bounding box or a convex hull. This coarseness is sufficient for many applications.

- Taking an image and predicting where and which type of objects appear
- Generally bounding box rather than specific pixels
- Multiple objects can overlap

Detection example: Opaque Regions in X-Rays

In this example the task is to detect opaque regions in lung X-ray images to provide a first indication for the physician who should make a diagnosis from the images. The used algorithm marks rectangles on region that are too opaque to be healthy.

[More details on Kaggle](#)

0.3.5 Other

- Unlimited possibilities [here](#)
- Horses to Zebras

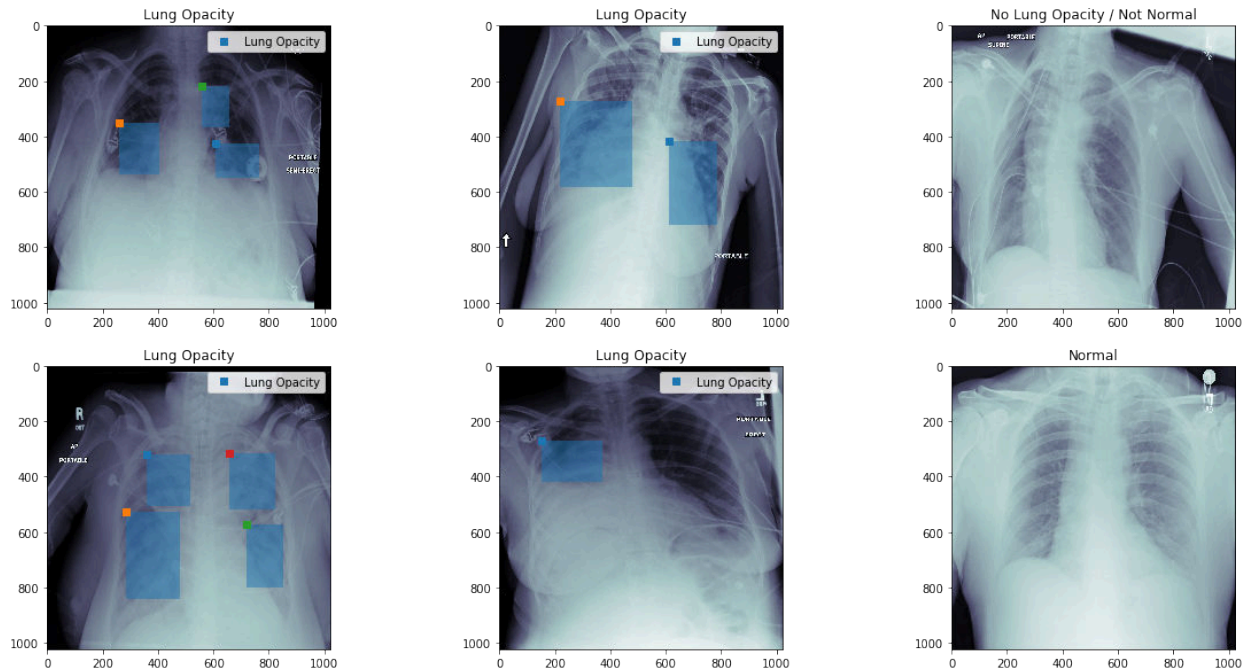


Fig. 7: Critical regions detected in lung radiographs.

Image Enhancement

- Denoising [Learning to See in the Dark](#)
- Super-resolution

0.4 Building your own data sets

Finally, we arrive at your data! As you already have seen, it is a time consuming and labor intense task to collect and prepare data.

- Very time consuming
- Not a lot of great tools
- Very problem specific

It is however important to have well-organized data for the analysis.

0.4.1 Code-free

Classification

- Organize images into folders

Regression

- Create an excel file (first column image name, next columns to regress)

Segmentation / Object Detection

- Take [FIJI](#) or any paint application and manually draw region to be identified and save it as a grayscale image

0.4.2 Software for data labelling

Free tools

- Classification/ Object Detection
- Classification (demo)
- Classification/ Detection
- Classification (Tinder for Brain MRI)

Example: annotation of spots

Spots are outliers in radiography and very annoying when the images are used for tomography.

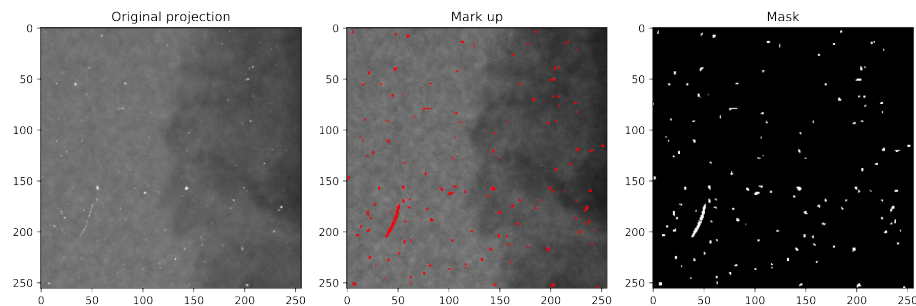


Fig. 1: Annotation of spot in a neutron radiograph.

- Image size 2048x2048
- Tools
 - Bitmap painting application
 - Drawing tablet
- Time to markup 8h

0.4.3 Simulations

A further way to increase training data is to build a model of the features you want to train on. This approach has the advantage that you know where to look for the features which means the tedious annotation task is reduced to a minimum. The work rather lies in building a reliable model that should reflect the characteristics of features you want to segment. Once a valid model is built, it is easy to generate masses of data under various conditions.

Simulations can be done using:

- Geometric models
- Template models
- Physical models

Both augmented and simulated data should be combined with real data.

Simulation examples

Another way to enhance or expand your dataset is to use simulations

- already incorporate realistic data (game engines, 3D rendering, physics models)
- 100% accurate ground truth (original models)
- unlimited, risk-free playability (driving cars in the world is more dangerous)

Examples

- P. Fuchs et al. - Generating Meaningful Synthetic Ground Truth for Pore Detection in Cast Aluminum Parts, iCT 2019, Padova
- Playing for Data: Ground Truth from Computer Games
- Self-driving car
- Learning from simulated data

0.5 Dataset Problems

Some of the issues which can come up with datasets are

- imbalance
- too few examples
- too homogenous
- and other possible problems

These lead to problems with the algorithms built on top of them.

0.5.1 Bias

Working with single sided data will bias the model towards this kind of data. This is a reason for the need to include as many corner cases as possible in the data. Biasing can easily happen when you have too few data to provide a statistically well founded training.

The gorilla example may sound fun but it can also upset people and in some cases the wrong descision can even cause inrepairable damage. Google's quick fix to the problem was to remove the gorilla category from their classifier. This approach may work for a trivial service like picture categorization tool, but yet again what if it is an essential category for the model?



Fig. 1: Mistakes that can happen due bias caused by insufficient training data.

The solution was to remove Gorilla from the category

A better solution to avoid biasing

Use training sets with more diverse people

The better solution to avoid biasing mistakes is to use a large data base with more variations one example is the [IBM Diverse Face Dataset](#). This face dataset not only provides great variation in people but also adds features to categorize the pictures even further. The figure below shows some samples from the face dataset with categories like:

- Accessories like eyeglasses and hats
- Different hair styles
- Face shapes
- Face expressions

IBM Diverse Face Dataset

Sample Images



Fig. 2: Use a database with more diverse people to avoid biasing.

0.5.2 Image data and labels

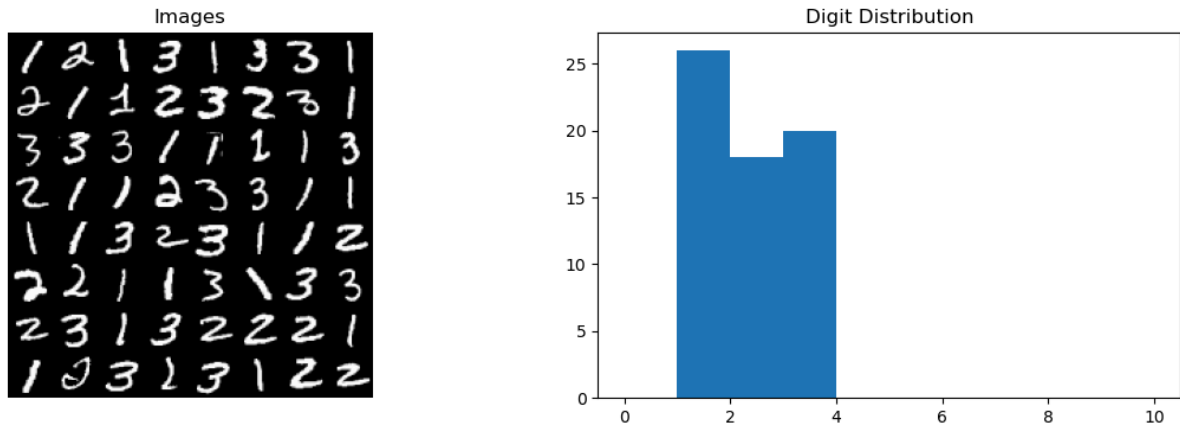
In the previous example with face pictures we started to look into categories of pictures. These pictures were provided with labels describing the picture content. The next dataset we will look at is the MNIST data set, which we already have seen a couple of times in this lecture.

In the example below we have extracted the numbers 1, 2, and 3. The histogram to the right shows the distribution of the numbers in the extracted data set.

```
(img, label), _ = mnist.load_data()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 4))
d_subset = np.where(np.in1d(label, [1, 2, 3]))[0]

# Visualization
ax1.imshow(montage2d(img[d_subset[:64], :, :]), cmap='gray', ax1.set_title('Images'),
            ax1.axis('off'))
ax2.hist(label[d_subset[:64]], np.arange(11), ax2.set_title('Digit
            Distribution');
```



0.5.3 Limited data

Machine learning methods require a lot of training data to be able to build good models that are able to detect the features they are intended to.

Different types of limited data:

- Few data points or limited amounts of images

This is very often the case in neutron imaging. The number of images collected during an experiment session is often very low due to the long experiment duration and limited amount of beam time. This makes it hard to develop segmentation and analysis methods for single experiments. The few data points problem can partly be overcome by using data from previous experiment with similar characteristics. The ability to recycle data depends on what you want to detect in the images.

- Unbalanced data

Unbalanced data means that the ratio between the data points with features you want detect and the total number data points is several orders of magnitude. E.g roots in a volume like the example we will look at later in this lecture. There is even a risk that the distribution of the wanted features is overlapped by the dominating background distribution.

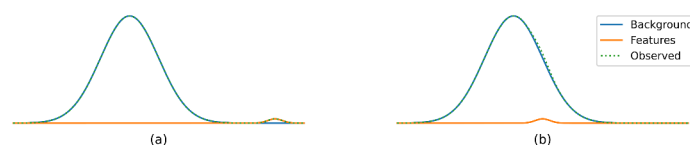


Fig. 3: Two cases of unbalanced data; (a) the classes are well separated and the feature class is clearly visible in the tail distribution of the background and (b) the feature class is embedded in the background making it hard to detect.

Case (a) can most likely be segmented using one of the many histogram based thresholding methods proposed in literature. Case (b) is much harder to segment as the target features have similar gray levels as the background. This case requires additional information to make segmentation possible.

- Little or missing training data

A complete set of training data contains both input data and labelled data. The input data is easy to obtain, it is the images you measured during your experiment. The labelled data is harder to get as it is a kind of chicken and egg problem. In particular, if your experiment data is limited. In that case, you would have to mark-up most of the available data to obtain the labelled data. Which doesn't make sense because

- then you'd already solved the task before even starting to train your segmentation algorithm.

- An algorithm based on learning doesn't improve the results, it only make it easier to handle large amounts of data.

0.6 Augmentation

Obtaining more experiment data is mostly relatively hard,

- Time in the lab is limited.
- Sample preparation is expensive.
- The number of specimens is limited.

Still, many supervised analysis methods require large data sets to perform reliably. A method to improve this situation is to use data augmentation. This means that you take the existing data and distorts it using different transformations or add features.

- Most groups have too little well-labeled data and labeling new examples can be very expensive.
- Additionally there might not be very many cases of specific classes.
- In medicine this is particularly problematic, because some diseases might only happen a few times in a given hospital and you still want to be able to recognize the disease and not that particular person.

0.6.1 Typical augmentation operations

Transformations

- Shift
- Zoom
- Rotation
- Intensity
- Normalization
- Scaling
- Color
- Shear

Further modifications

- Add noise
- Blurring

0.6.2 Some augmentation examples

The figure below shows some examples of augmentations of the same image. You can also add noise and modulate the image intensity to increase the variations further.

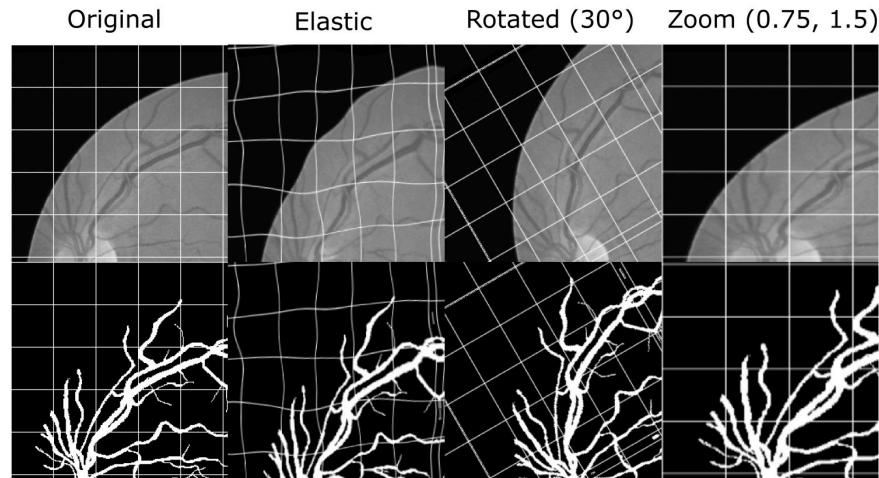


Fig. 1: A retinal image modified using different augmentation techniques (source: <https://drive.grand-challenge.org/DRIVE/>) prepared by Gian Guido Parenza.

Retina images from [DRIVE](#) prepared by Gian Guido Parenza.

0.6.3 Limitations of augmentation

What transformations are normal in the images?

- CT images usually do not get flipped (the head is always on the top)
- The values in CT images have a physical meaning (Hounsfield unit), → scaling them changes the interpretation.

How much distortion is too much?

- Can you still recognize the features?
- Is the distortion within natural bounds?

0.6.4 Keras ImageDataGenerator

[Augmentation demonstration](#)

[Help page of the data generator](#)

```
ImageDataGenerator(
    ['featurewise_center=False', 'samplewise_center=False', 'featurewise_std_
    ↪normalization=False', 'samplewise_std_normalization=False', 'zca_whitening=False',
    ↪'zca_epsilon=1e-06', 'rotation_range=0.0', 'width_shift_range=0.0', 'height_shift_
    ↪range=0.0', 'shear_range=0.0', 'zoom_range=0.0', 'channel_shift_range=0.0', "fill_
    ↪mode='nearest'", 'cval=0.0', 'horizontal_flip=False', 'vertical_flip=False',
    ↪'rescale=None', 'preprocessing_function=None', 'data_format=None'],
)
```

(continues on next page)

(continued from previous page)

```

Docstring:
Generate minibatches of image data with real-time data augmentation.

# Arguments
featurewise_center: set input mean to 0 over the dataset.
samplewise_center: set each sample mean to 0.
featurewise_std_normalization: divide inputs by std of the dataset.
samplewise_std_normalization: divide each input by its std.
zca_whitening: apply ZCA whitening.
zca_epsilon: epsilon for ZCA whitening. Default is 1e-6.
rotation_range: degrees (0 to 180).
width_shift_range: fraction of total width, if < 1, or pixels if >= 1.
height_shift_range: fraction of total height, if < 1, or pixels if >= 1.
shear_range: shear intensity (shear angle in degrees).
zoom_range: amount of zoom. if scalar z, zoom will be randomly picked
            in the range [1-z, 1+z]. A sequence of two can be passed instead
            to select this range.
channel_shift_range: shift range for each channel.
fill_mode: points outside the boundaries are filled according to the
           given mode ('constant', 'nearest', 'reflect' or 'wrap'). Default
           is 'nearest'.
           Points outside the boundaries of the input are filled according to the given_
mode:
    'constant': kkkkkkkk|abcd|kkkkkkkk (cval=k)
    'nearest':  aaaaaaaaa|abcd|dddddddd
    'reflect':  abcd dcba|abcd|dcbaabcd
    'wrap':     abcdabcd|abcd|abcdabcd
cval: value used for points outside the boundaries when fill_mode is
      'constant'. Default is 0.
horizontal_flip: whether to randomly flip images horizontally.
vertical_flip: whether to randomly flip images vertically.
rescale: rescaling factor. If None or 0, no rescaling is applied,
         otherwise we multiply the data by the value provided. This is
         applied after the `preprocessing_function` (if any provided)
         but before any other transformation.
preprocessing_function: function that will be implied on each input.
                        The function will run before any other modification on it.
                        The function should take one argument:
                        one image (Numpy tensor with rank 3),

```

A Keras ImageDataGenerator example

There are quite many degrees of freedom to use the ImageDataGenerator. The generator is given all boundary condition at initialization time. Below you see an example of how it can be initialized.

```
from keras.datasets import mnist
```

```

from tensorflow.keras.preprocessing.image import ImageDataGenerator

img_aug = ImageDataGenerator(
    featurewise_center = False,
    samplewise_center  = False,
    zca_whitening      = False,
    zca_epsilon        = 1e-06,
    rotation_range      = 30.0,

```

(continues on next page)

(continued from previous page)

```

width_shift_range    = 0.25,
height_shift_range   = 0.25,
shear_range          = 0.25,
zoom_range           = 0.5,
fill_mode            = 'nearest',
horizontal_flip       = False,
vertical_flip        = False
)

```

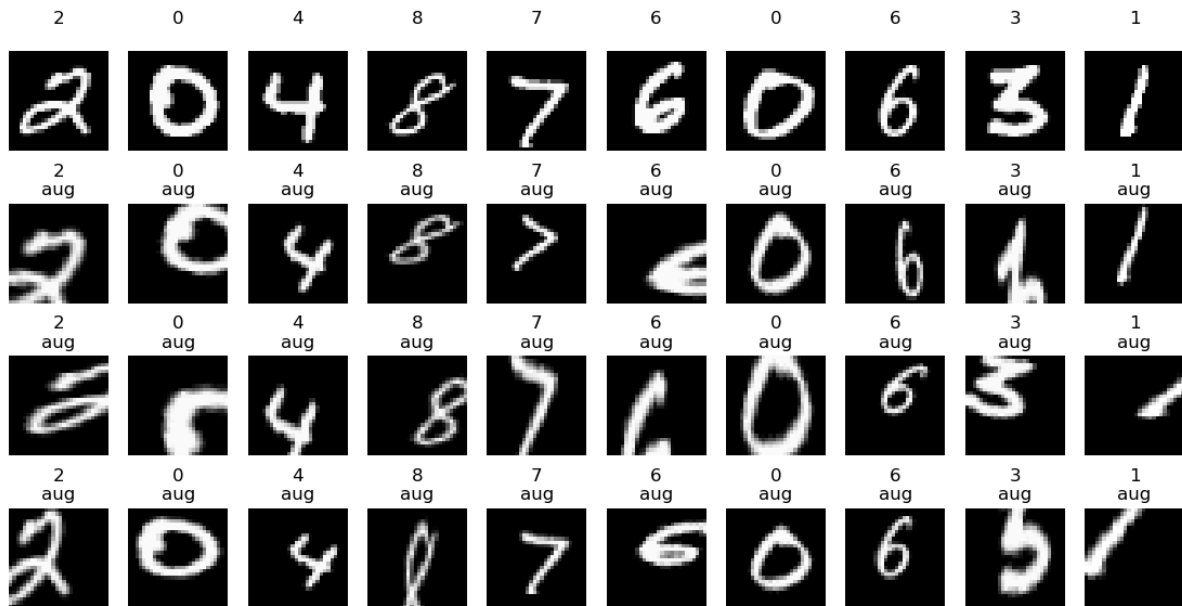
0.6.5 Augmenting MNIST images

Even something as simple as labeling digits can be very time consuming (maybe 1-2 per second).

```

import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist
%matplotlib inline

```



0.6.6 A larger open data set

We can use a more exciting dataset to try some of the other features in augmentation.

The CIFAR-10 dataset consists of 60000 32x32 colour images in 10 classes, with 6000 images per class. There are 50000 training images and 10000 test images.

Here are some examples from the [CIFAR10](#) dataset

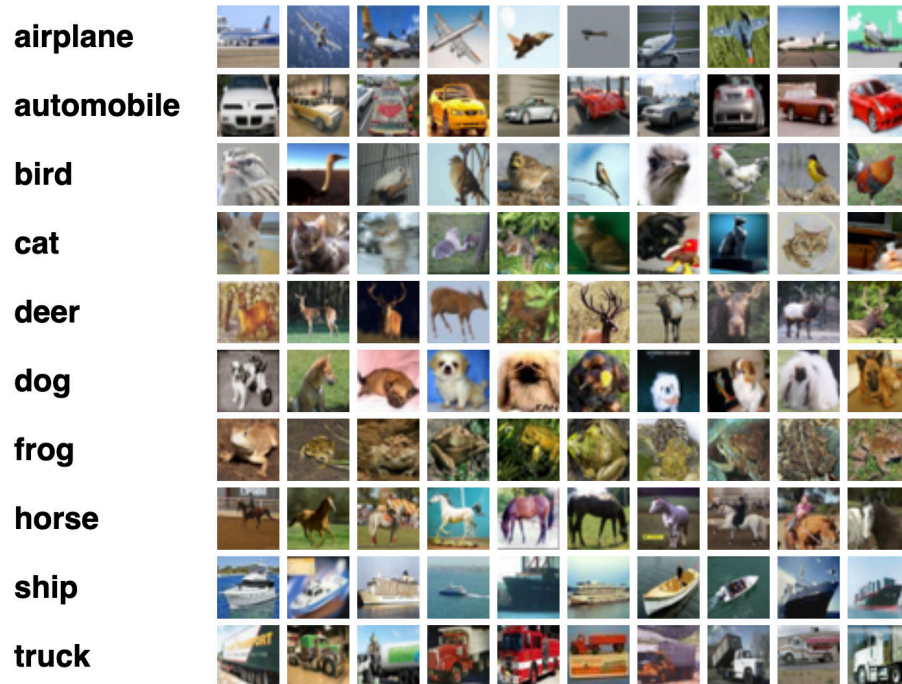


Fig. 2: Some examples form the CIFAR-10 dataset

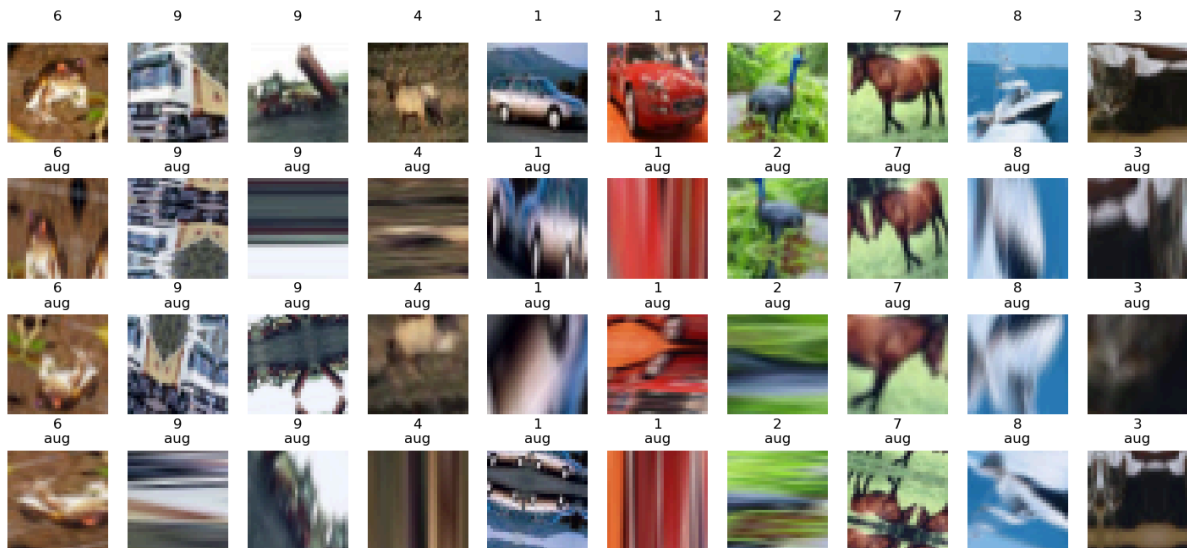
Augmenting CIFAR10 images

```
from keras.datasets import cifar10
(img, label), _ = cifar10.load_data()
```

```
img_aug = ImageDataGenerator(
    featurewise_center = True,
    samplewise_center  = False,
    zca_whitening       = False,
    zca_epsilon         = 1e-06,
    rotation_range      = 30.0,
    width_shift_range   = 0.25,
    height_shift_range  = 0.25,
    channel_shift_range = 0.25,
    shear_range         = 0.25,
    zoom_range          = 1,
    fill_mode           = 'reflect',
    horizontal_flip     = True,
    vertical_flip       = True
)
```

Running the CIFAR augmentation

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist
%matplotlib inline
```



0.7 Baselines

A baseline is

- a simple,
- easily implemented and understood model
- that illustrates the problem
- and the ‘worst-case scenario’ for a model that learns nothing (some models will do worse, but these are especially useless).

Why is this important?

0.7.1 Baseline model example

I have a model that is >99% accurate for predicting breast cancer:

DoIHaveBreastCancer(Age, Weight, Race) = No!

0.7.2 The dummy classifier

Let's train the dummy classifier with some values related to healthy and cancer sick patients. Measurements values 0,1, and 2 are healthy while the value 3 has cancer. We train the classifier with the strategy that the most frequent class will predict the outcome.

```
from sklearn.dummy import DummyClassifier

dc = DummyClassifier(strategy='most_frequent')

dc.fit([0, 1, 2, 3],
      ['Healthy', 'Healthy', 'Healthy', 'Cancer'])
```

```
DummyClassifier(strategy='most_frequent')
```

Testing the outcome of the classifier

```
for idx in [0,1,3,100] :
    print('Prediction for {0} is {1}'.format(idx,dc.predict([idx])[0]))
```

```
Prediction for 0 is Healthy
Prediction for 1 is Healthy
Prediction for 3 is Healthy
Prediction for 100 is Healthy
```

With these few lines we test what happens when we provide some numbers to the classifier. The numbers are

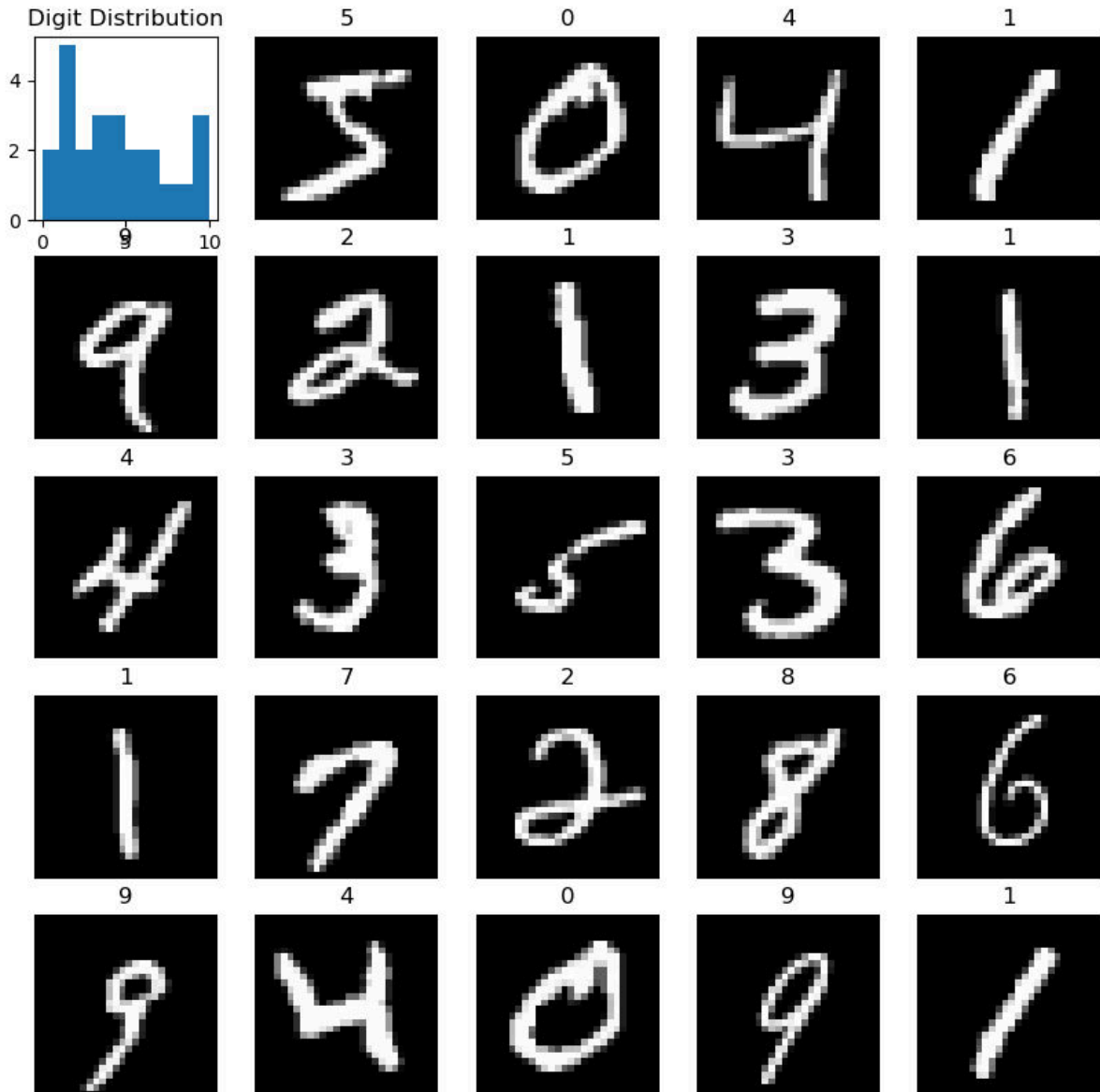
- 0 and 1, which are expected to be healthy
- 3 , which has cancer
- 100, unknown to the model

So, the classifier tells us that all values are from healthy patients... not really good! The reason is that it was told to tell us the category of the majority, which is that the patient is healthy.

0.7.3 Try dummy classifier on MNIST data

The previous basic problem showed us how the dummy classifier work. Now we want to use it with the handwritten numbers in the MNIST dataset. The first step is to load the data and check how the distribution of numbers in the data set using a histogram.

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist
from skimage.util import montage as montage2d
%matplotlib inline
```



Let's train the model...

Now we want to train the model with our data. Once again we use the *most frequent* model. The training is done in the first 24 images in the data set. The fitting requires that we provide the images with numbers and their associated labels telling the model how to interpret the image.

```
dc = DummyClassifier(strategy='most_frequent')
dc.fit(img[:24], label[:24])
```

```
DummyClassifier(strategy='most_frequent')
```

A basic test

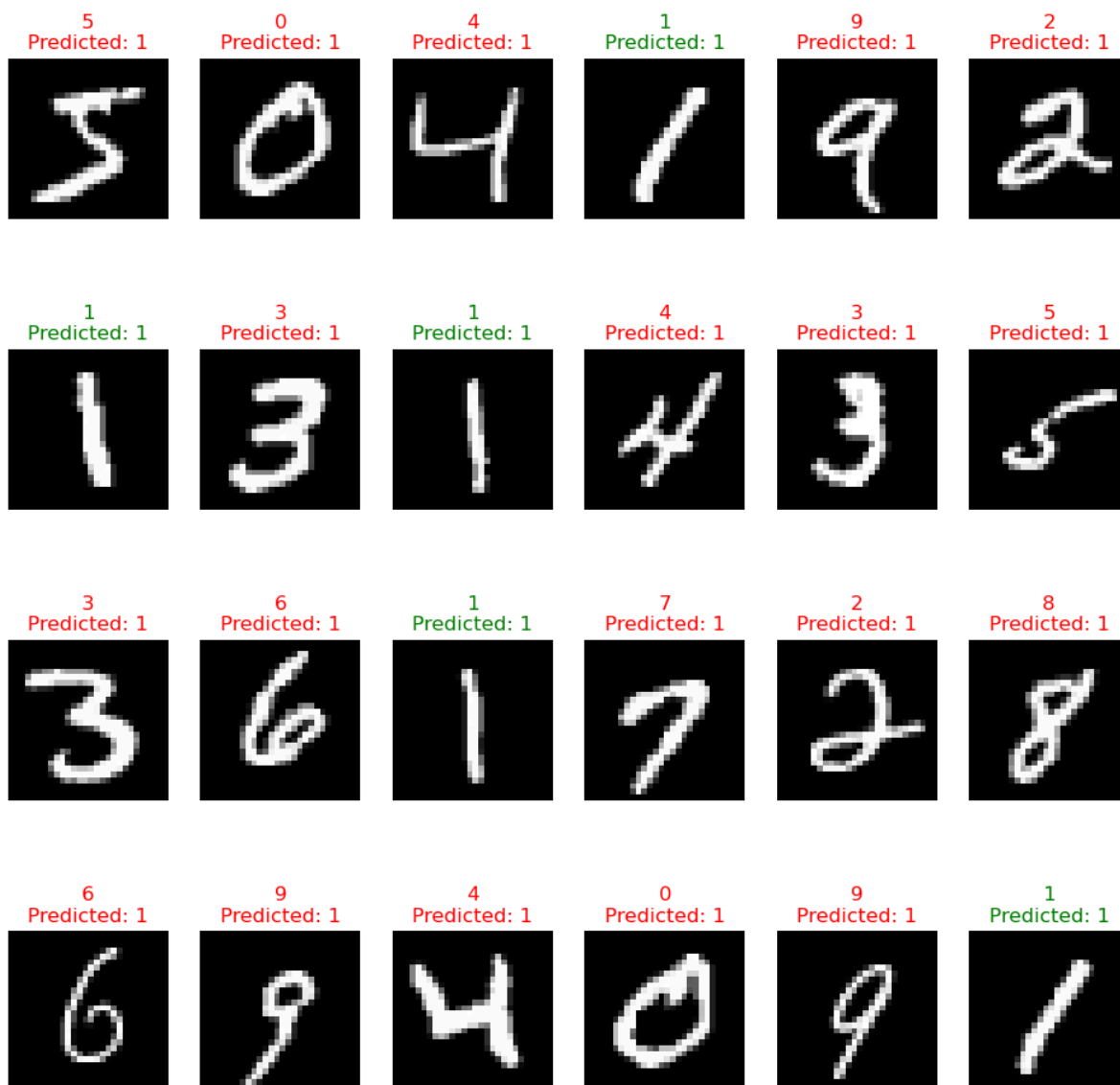
In the basic test, we provide the first ten images and hope to get predictions which numbers they represent.

```
dc.predict(img[0:10])
```

```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=uint8)
```

Test on the images

Let's see how good these predictions really are by showing the images along with their labels and the prediction of the trained model.

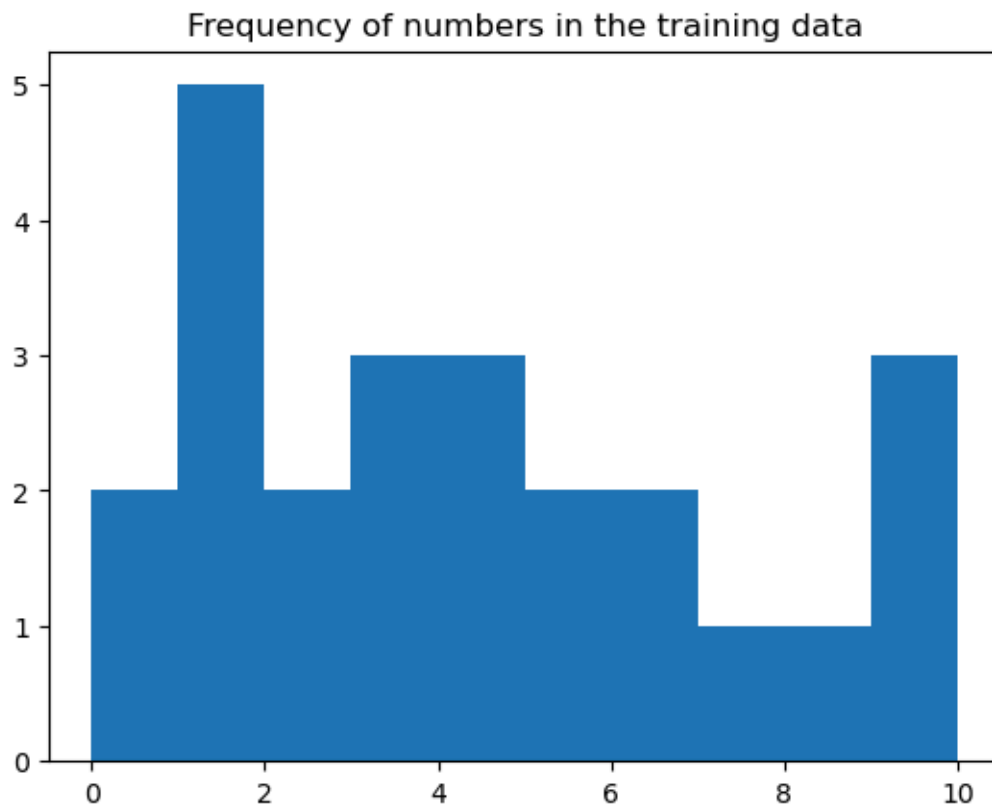


... why are all predictions = 1?

The result of the basic classifier was quite disappointing. It told us that all ten images contained the number '1'. Now, why is that?

This can be explained by looking at the label histogram:

```
plt.hist(label[:24], np.arange(11)); plt.title('Frequency of numbers in the training_  
data');
```



Here, we see that there are most '1's in the training data. We have been using the *most frequent* model for training the classifier therefore the response '1' is the only answer the model can give us.

0.7.4 Nearest Neighbor

A better baseline

This isn't a machine learning class and so we won't dive deeply into other methods, but nearest neighbor is often a very good baseline (that is also very easy to understand). You basically take the element from the original set that is closest to the image you show.

Figure from J. Russ, Image Processing Handbook

You can make the method more robust by using more than one nearest neighbor (hence K nearest neighbors), but that we will cover later in the supervised methods lecture.

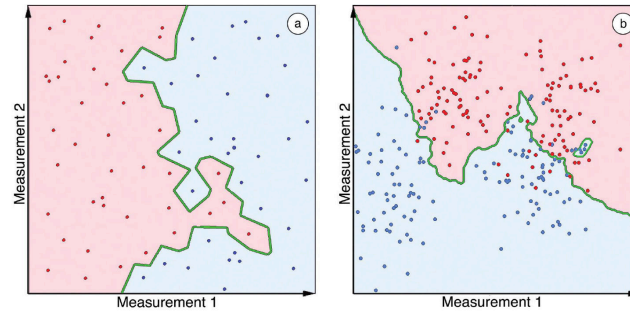


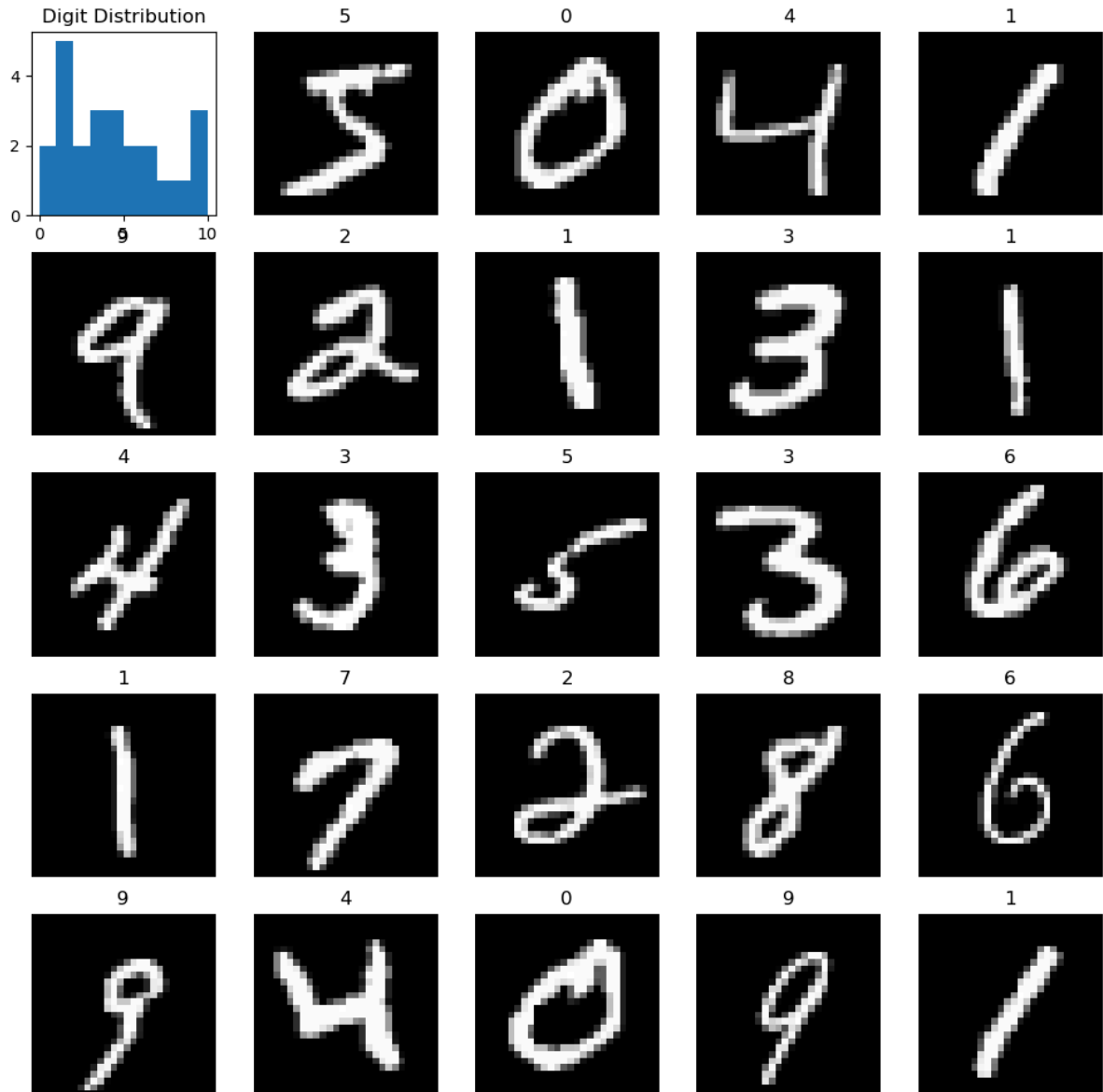
Figure 12.58 The irregular boundary between two classes using k -nearest-neighbor classification:
(a) $k = 1$; (b) $k = 9$.

Fig. 1: Examples of the k-nearest neighbors classifier (Figure from J. Russ, Image Processing Handbook).

Let's load the data again...

Let's come back to the MNIST numbers again. This time, we will try the k-nearest neighbors as baseline and see if we can get a better result than with the dummy classifier with majority voting.

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist
from skimage.util import montage as montage2d
%matplotlib inline
```



Training k-nearest neighbors

The training of the k-nearest neighbors consists of filling feature vectors into the model and assign each vector to a class. But images are not vectors... so what we do is to rearrange the $N \times M$ images into a vector with the dimensions $M \cdot N \times 1$.

```
from sklearn.neighbors import KNeighborsClassifier
neigh_class = KNeighborsClassifier(n_neighbors=1)

N = 24
neigh_class.fit(img[:N].reshape((N, -1)), label[:N])
```

```
KNeighborsClassifier(n_neighbors=1)
```

Predict on a few images

The prediction of which class an image belongs to is done by reshaping the input image into a vector in the same manner as for the training data. Now we will compare the input vector u to all the vectors in the trained model v_i by computing the Euclidean distance between the vectors. This can easily be done by the inner product of the two vectors:

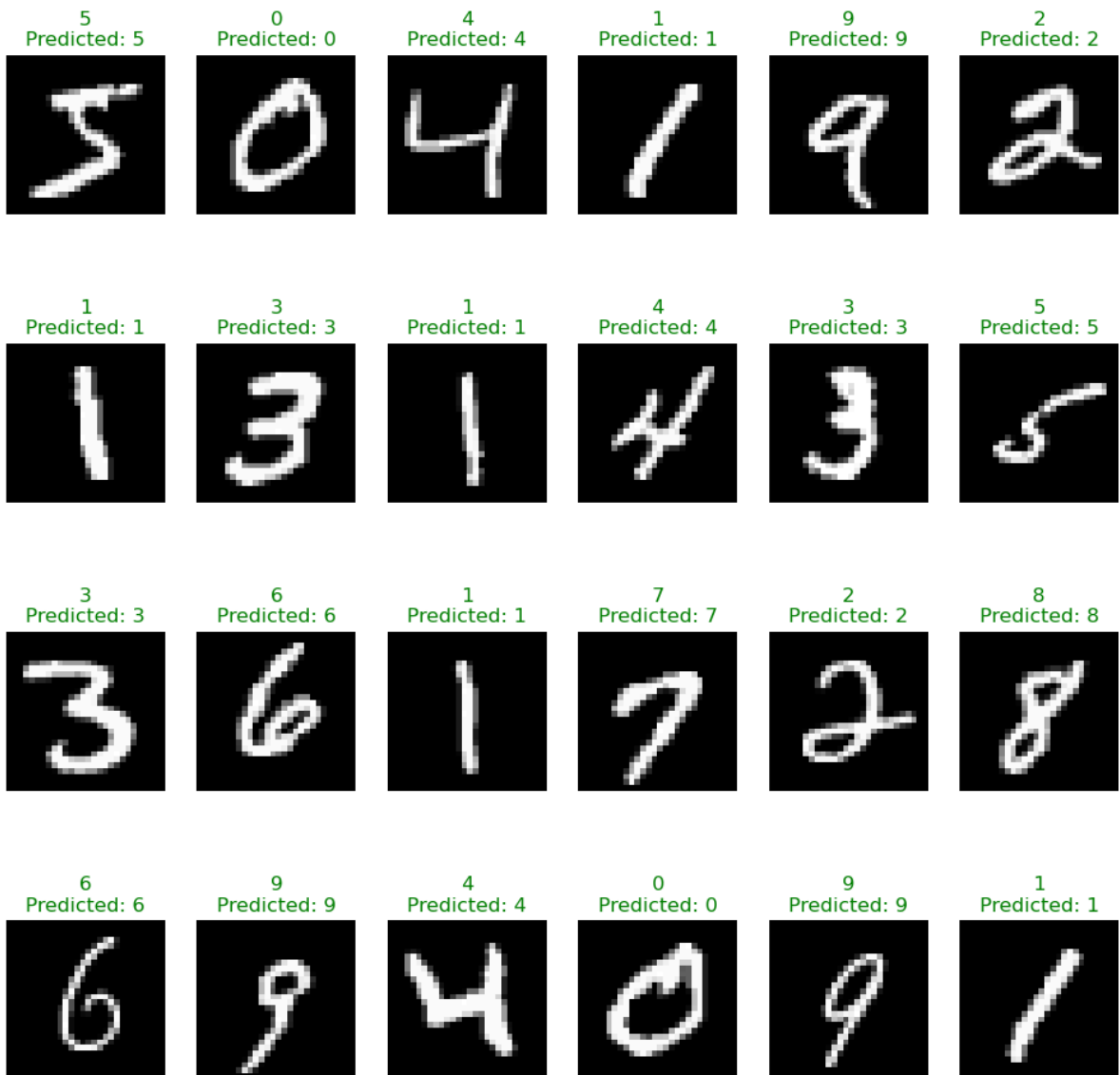
$$D_i = (v_i - u)^T \cdot (v_i - u) = \text{scalar}$$

The class is chosen by the model vector that is closest to the input vector, i.e. having the smallest D_i . This calculations are done for you as a black box in the `KNeighborsClassifier`, you only have to reshape the images into the right format.

```
neigh_class.predict(img[0:10].reshape((10, -1)))
```

```
array([5, 0, 4, 1, 9, 2, 1, 3, 1, 4], dtype=uint8)
```

Compare predictions with the images



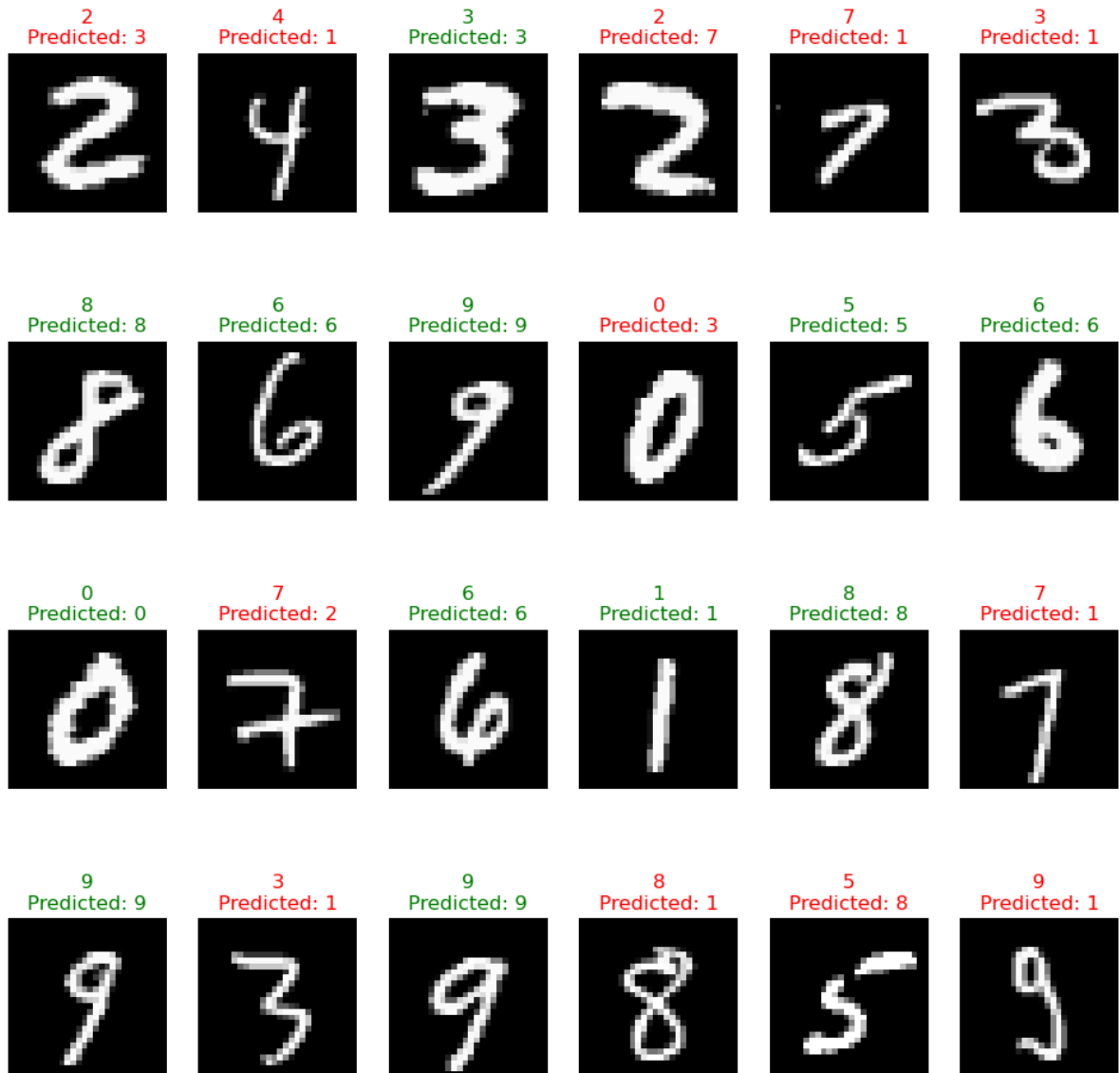
Wow, 100% correct!

100% for a baseline !!?

Wow the model works really really well, it got every example perfectly.

What we did here (a common mistake) was evaluate on the same data we ‘trained’ on which means the model just correctly recalled each example. This is natural as there is always an image that gives the distance $D_i = 0$.

Now, if we try it on new images we can see the performance drop but still a somewhat reasonable result.



0.7.5 How good is good?

From the previous example, we saw that the classify doesn't really reach the 100% accuracy on unseen data, but rather makes a mistake here or there. Therefore we need to quantify how good it really is to be able to compare the results with other algorithms. We will cover more tools later in the class but now we will show the accuracy and the confusion matrix for our simple baseline model to evaluate how well it worked.

Confusion Matrix

The confusion matrix is a kind of histogram where you count the number of predicted occurrences for each actual label. This gives us an idea about the classifier performance.

We show which cases were most frequently confused

n=165	Predicted TRUE	Predicted FALSE
Actual TRUE	50	10
Actual FALSE	5	100

This is only a simple matrix for the two cases *true* and *false*. The matrix does however grow with the number of classes in the data set. It is always a square matrix as we have the same number of actual classes as we have predicted classes.

Confusion matrix for the MNIST classification

We saw that the k-nearest neighbors did a couple of missclassifications on the unseen test data. Now is the question how many mistakes it really does and how many correct labels it assigned. If compute the confusion matrix for this example, we will get a 10x10 matrix i.e. one for each class in the data set.

```
import seaborn as sns
import pandas as pd
def print_confusion_matrix(confusion_matrix, class_names, figsize = (10,7),
    ↪fontsize=14):
    """Prints a confusion matrix, as returned by sklearn.metrics.confusion_matrix, as
    ↪a heatmap.

    Stolen from: https://gist.github.com/shaypal5/94c53d765083101efc0240d776a23823

    Arguments
    -----
    confusion_matrix: numpy.ndarray
        The numpy.ndarray object returned from a call to sklearn.metrics.confusion_
    ↪matrix.
        Similarly constructed ndarrays can also be used.
    class_names: list
        An ordered list of class names, in the order they index the given confusion_
    ↪matrix.
    figsize: tuple
        A 2-long tuple, the first value determining the horizontal size of the
    ↪ouputted figure,
        the second determining the vertical size. Defaults to (10,7).
    fontsize: int
        Font size for axes labels. Defaults to 14.

    Returns
    -----
    matplotlib.figure.Figure
        The resulting confusion matrix figure
    """
    df_cm = pd.DataFrame(
        confusion_matrix, index=class_names, columns=class_names,
    )
    fig, ax1 = plt.subplots(1, 1, figsize=figsize)
```

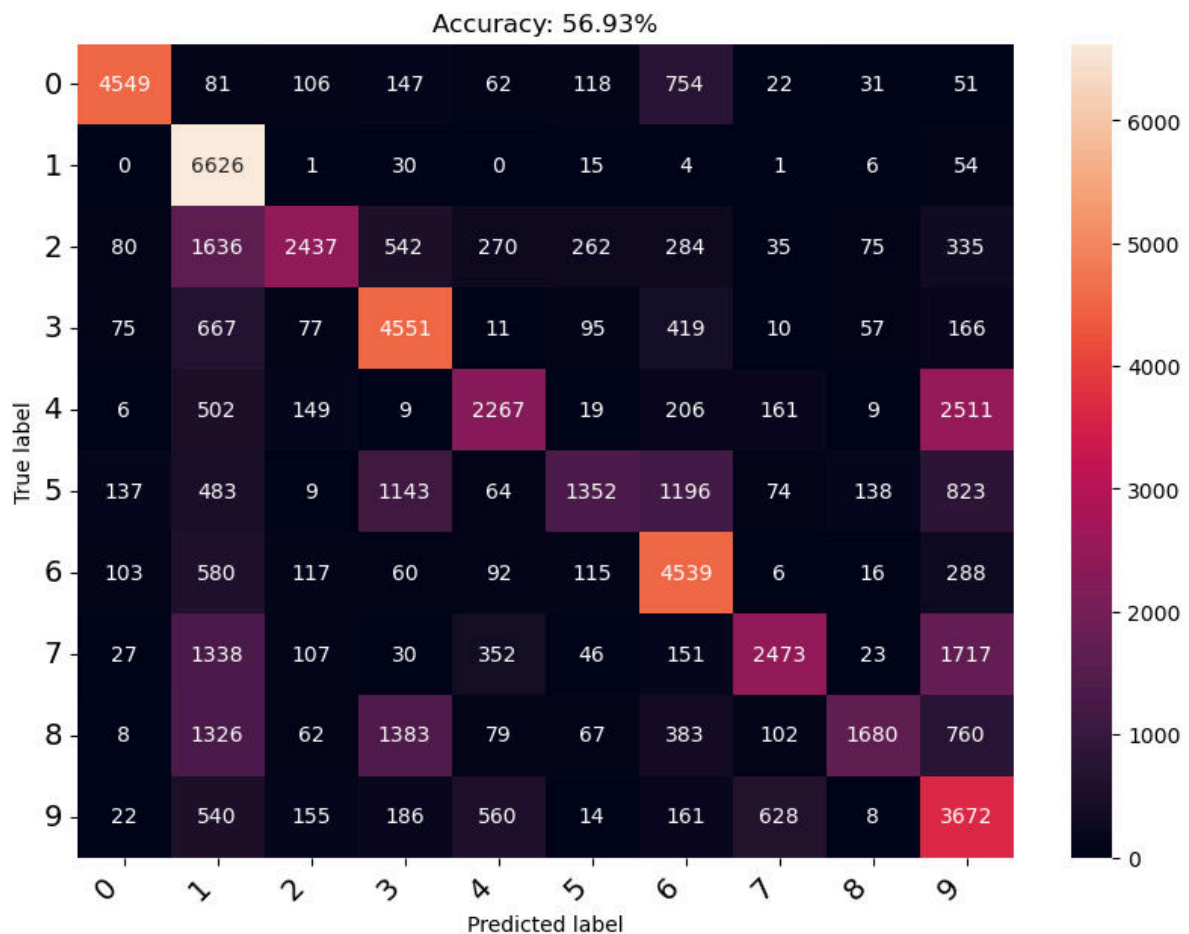
(continues on next page)

(continued from previous page)

```

try:
    heatmap = sns.heatmap(df_cm, annot=True, fmt="d")
except ValueError:
    raise ValueError("Confusion matrix values must be integers.")
heatmap.yaxis.set_ticklabels(heatmap.yaxis.get_ticklabels(), rotation=0, ha='right',
    ↪', fontsize=fontsize)
heatmap.xaxis.set_ticklabels(heatmap.xaxis.get_ticklabels(), rotation=45, ha=
    ↪'right', fontsize=fontsize)
plt.ylabel('True label')
plt.xlabel('Predicted label')
return ax1

```



In this confusion matrix we see that some numbers are easier to classify than others. Some examples are:

- The '0' seems to be hard to confuse with other numbers.
- Many images from all categories are falsely assigned to the '1'
- The number '4' is more probable to be assigned a label '9' than the '4'

This experiment was done with a very limited training data set. You can experiment with more neighbors and more training data to see what improvement that brings. In all, there are 60000 images with digits in the data set.

0.8 Data frames - managing feature tables

0.8.1 Our workflow

Analysing data results in a reduction of images which are collections of large arrays of data into tables of measured features. The feature metrics is the information we want to analyse in our scientific investigation. These steps can roughly be broken down into the following three steps.

- Image analysis
- Feature selection and analysis
- Presentation

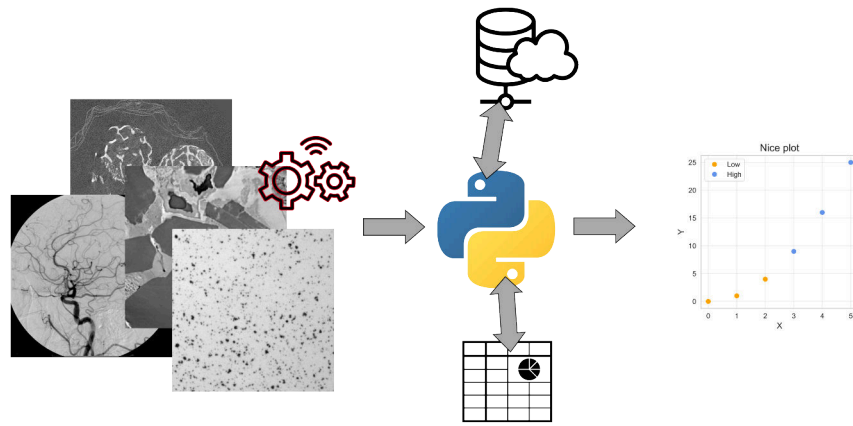


Fig. 1: Different destinations of the processed data.

The question is how to best store data from different step to facilitate an efficient analysis workflow. So,

How do we store the features while working with the data?

0.8.2 How do we store the features?

- Our analysis results in a collection of features
- We need a structured storage solution
- We want to perform further analysis

The data from the image analysis consists of lists of values for the measured features. These value list are however only the base for the detailed statistical analysis and visualization. A collection of arrays with values are difficult to use in a structured analysis, and it must be reimplemented for each analysis case.

Operations on the feature data

We often want to perform the following operations on the data tables in the progress of the analysis:

- Counting
- Statistics
- Selections
- Transforms
- Visualization

Python offers different options

There are several build in features for working with data in python. These are essentially classic data structures for most programming languages.

- Arrays per feature
- List of data structures/dictionaries

Problem

The problem with these data structures is that they don't provide many preimplemented functions to work with. Therefore, With custom storage we have to implement functions for each operation:

- Time consuming
- Little flexibility
- Error prone

And it also requires quite some programming skill to implement these function in a reusable and stable manner. Many students have already spent waste amounts of time to implement their workflow from a low level. These effort are often a reimplementation of what has already been done by many other students, with slight variations to adapt to the problem at hand.

0.8.3 Introducing data frames

We have already seen data frames in action but never formally introduced them...

A data frame is

- A data container
- Organized into columns and rows
- Similar to a spread sheet table
- Takes any data in the columns

With the data frame you can

- Apply filters for selection
- Sort the rows
- Perform arithmetics
- Compute statistics
- Read and store into files and databases

Pandas documentation [Getting started with Pandas](#)

0.8.4 Create a data frame

There are different ways to create a data frame:

- From a dict
- From a data file
- From numpy arrays

First we have import pandas:

```
import pandas as pd
```

Which gives us access to pandas data frames and functions to work with them.

Read from a spreadsheet (csv)

Sometimes the features have been extracted elsewhere and stored in a file, e.g. CSV

The file extension CSV is an abbreviation for Comma Separated Values. This is a basic file format which is easy to read and write from a processing workflow. It is also human readable as the values are written in clear text separated by commas or tabs. Spreadsheet programs can also work with CSV files.

The data set we read here is from a phenotype study on bone mechanics.

```
pheno = pd.read_csv('../Exercises/08-Statistics/phenoTable.csv')
pheno.sample(5)
```

	BMD	MECHANICS_STIFFNESS	CORT.DTO__C_TH	CORT.DTO__C_TH_SD	\
487	0.0384	73.878304	0.209730	0.023124	
300	0.0340	76.715228	0.188596	0.018494	
962	0.0331	59.376754	0.164399	0.023096	
239	0.0349	52.222113	0.188559	0.027589	
184	0.0347	59.704646	0.176771	0.014658	
	CORT.MOM__J	CT_TH_RAD	CT_TH_RAD_STD	CANAL_VOLUME	CANAL_COUNT \
487	0.181331	83.238684	12.436912	26212.629973	263.0
300	0.184996	96.418538	26.579545	120621.794963	150.0
962	0.137650	73.356389	10.991060	19182.687798	190.0
239	0.116116	84.905764	21.543230	44206.688124	193.0
184	0.132583	77.614988	16.732023	11403.226871	104.0
	CANAL_DENSITY	...	CANAL_THETA	CANAL_THETA_CV	CANAL_PCA1 \

(continues on next page)

(continued from previous page)

487	268.756482	...	57.463630	0.361793	285.580430	
300	165.491535	...	45.945449	0.528751	202.403328	
962	234.803718	...	57.472957	0.352902	259.125770	
239	221.404233	...	55.678224	0.394700	251.623034	
184	139.736893	...	56.045035	0.360029	334.096851	

	CANAL_PCA1_CV	CANAL_PCA2	CANAL_PCA2_CV	CANAL_PCA3	CANAL_PCA3_CV	\
487	1.151501	80.843025	1.015263	35.147216	0.838448	
300	0.680364	64.606619	0.686461	31.133847	0.602466	
962	1.045595	63.174166	0.834666	29.694688	0.491750	
239	0.769725	78.183283	0.911765	33.703465	0.639786	
184	0.962678	65.072767	1.189292	26.044027	0.548526	

	FEMALE	ID
487	0	1339
300	0	1042
962	1	2504
239	0	813
184	1	745

[5 rows x 35 columns]

When we look at the data using the sample function you get a random sample of N items, the rows are not even sorted according to their row indices. The content is already formatted as a table and the column labels were extracted from the first line of the CSV file.

Saving works similarly:

```
pheno.to_csv('pheno2.csv')
```

Create a data frame using dicts

We create the data frame as the features are produced.

Loading data from a file is often good if your processing runs a cascade of scripts that write to result files. A different approach to fill the data frame to do it while the processing script is running. The best way is then to create dictionaries with the measured data. This can be done in two ways.

1. Create a list with dictionaries for each analysis run.
2. Create lists for each parameter which then are combined into a dictionary.

In the following example we create a list of dictionaries by first creating the list and then create a dataframe based on the list.

```
dl = []
for i in np.arange(0,10) :
    dl.append({'position' : i, 'sine' : np.sin(i)})
print(dl)
```

```
[{'position': 0, 'sine': 0.0}, {'position': 1, 'sine': 0.8414709848078965}, {
↪ 'position': 2, 'sine': 0.9092974268256817}, {'position': 3, 'sine': 0.
↪ 1411200080598672}, {'position': 4, 'sine': -0.7568024953079283}, {'position': 5,
↪ 'sine': -0.9589242746631385}, {'position': 6, 'sine': -0.27941549819892586}, {
↪ 'position': 7, 'sine': 0.6569865987187891}, {'position': 8, 'sine': 0.
↪ 9893582466233818}, {'position': 9, 'sine': 0.4121184852417566}]
```

```
df = pd.DataFrame(d1)

df.head(5)
```

	position	sine
0	0	0.000000
1	1	0.841471
2	2	0.909297
3	3	0.141120
4	4	-0.756802

Here, we used a different way to inspect the dataframe. The function head lists the N first rows of the dataframe. You can also look at the last rows by using tail.

An alternative...

You may already have numpy arrays with the data.

This is the second method to create a dataframe. Bundling lists or numpy arrays into a dictionary.

```
x = np.linspace(0, 2*np.pi, 100)
sine = np.sin(x)
dd = {'position' : x, 'sine' : sine}

ddf = pd.DataFrame(dd)
ddf.head(5)
```

	position	sine
0	0.000000	0.000000
1	0.063467	0.063424
2	0.126933	0.126592
3	0.190400	0.189251
4	0.253866	0.251148

The resulting dataframe is the same as for the case of a list of dictionaries. Which style you use to build your dataframes depends on what suits your workflow best.

Working with columns

Add a new column

When we start working on the data, we may need to add a column

```
df['cosine'] = np.cos(df['position'])
df['sum'] = df['sine'] + df['cosine']
df['positive'] = 0 < df['sine']
df.head()
```

	position	sine	cosine	sum	positive
0	0	0.000000	1.000000	1.000000	False
1	1	0.841471	0.540302	1.381773	True
2	2	0.909297	-0.416147	0.493151	True
3	3	0.141120	-0.989992	-0.848872	True
4	4	-0.756802	-0.653644	-1.410446	False

Select some rows with content filtering

A great advantage of dataframes is that you can select contents based some criteria. The example below shows filtering that you can apply to the dataframe. The result can be assigned to either a new dataframe or a new column in the current dataframe.

```
df2=df[0<df['sine']]
df3=df[df['positive']]
df2.head(5)
```

	position	sine	cosine	sum	positive
1	1	0.841471	0.540302	1.381773	True
2	2	0.909297	-0.416147	0.493151	True
3	3	0.141120	-0.989992	-0.848872	True
7	7	0.656987	0.753902	1.410889	True
8	8	0.989358	-0.145500	0.843858	True

Rename column titles

Note: Here is also a different way to create a data frame with dicts and lists.

Let's first create a small test dataframe with the two columns 'A' and 'B'

```
df3 = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
df3.head()
```

	A	B
0	1	4
1	2	5
2	3	6

The current column names may not be so intuitive so we want to rename them. This is done with the function 'rename' that takes a dictionary as argument. The dictionary contains the current column titles as item keys and the new name as values.

```
df3 = df3.rename(columns={"A": "hej", "B": "hopp"})
df3
```

	hej	hopp
0	1	4
1	2	5
2	3	6

0.8.5 Statistics with a data frame

You can easily compute statistics on a data frame:

- mean
- std
- min
- max
- median

```
df.mean()
```

```
position    4.500000
sine        0.195521
cosine      0.042162
sum         0.237683
positive    0.600000
dtype: float64
```

Summarizing the statistics of dataframe

You can also create a quick summary of the dataframe statistics to get an overview.

```
df.describe()
```

	position	sine	cosine	sum
count	10.000000	10.000000	10.000000	10.000000
mean	4.500000	0.195521	0.042162	0.237683
std	3.02765	0.693076	0.765706	1.009325
min	0.000000	-0.958924	-0.989992	-1.410446
25%	2.250000	-0.209562	-0.594269	-0.631200
50%	4.500000	0.276619	0.069081	0.586953
75%	6.750000	0.795350	0.700502	0.960965
max	9.000000	0.989358	1.000000	1.410889

0.8.6 Statistics of filtered data

Compute

- the standard deviation
- for the “sine” column
- where the rows have $0 < \text{sum}$

```
df.head()
df[0<df["sum"]]['sine'].std()
```

```
0.5297967209946162
```

Statistics using grouping

Filtering works well for few categories, but for many categories some smarter tricks can be used:

The following call groups the rows based on values in the column ‘positive’ and computes the average of the other columns where the current value in positive is matched.

```
# compute category statistics
group_name='positive'
data_agg = df.groupby(group_name).mean()

data_agg
```

	position	sine	cosine	sum
positive				
False	3.75	-0.498786	0.397547	-0.101238
True	5.00	0.658392	-0.194761	0.463631

Too cluttered?

We may not be interested in the all columns, maybe want to add variance.

This example computes the mean and var for the sine column the the positive column is true and false.

```
# compute category statistics
group_name='positive'
value_name='sine'
data_agg = df.groupby(group_name).agg({value_name: ['mean', 'var']}).reset_index()
data_agg.columns = data_agg.columns.get_level_values(1)

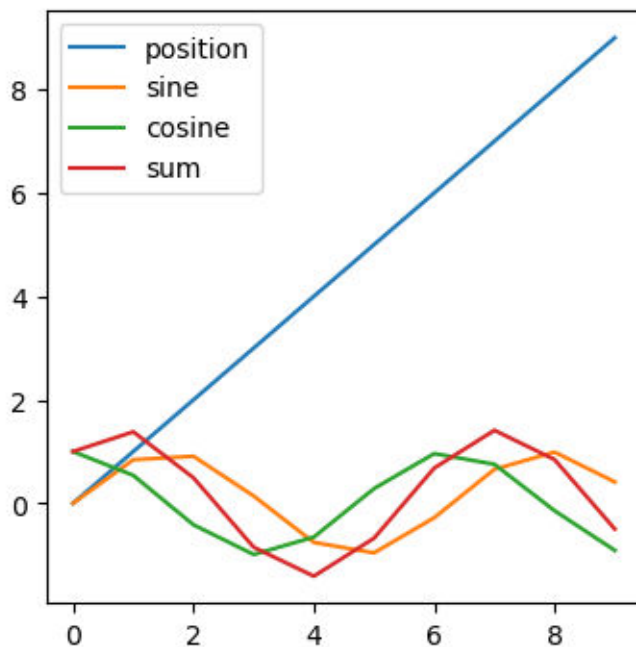
data_agg
```

		mean	var
0	False	-0.498786	0.191737
1	True	0.658392	0.106847

0.8.7 Visualizing the contents of a data frame

In addition to all other plotting options, Pandas supports some basic plotting functionality

```
fig,ax=plt.subplots(1,1,figsize=(4,4))
df.plot(ax=ax);
```



Selective plotting

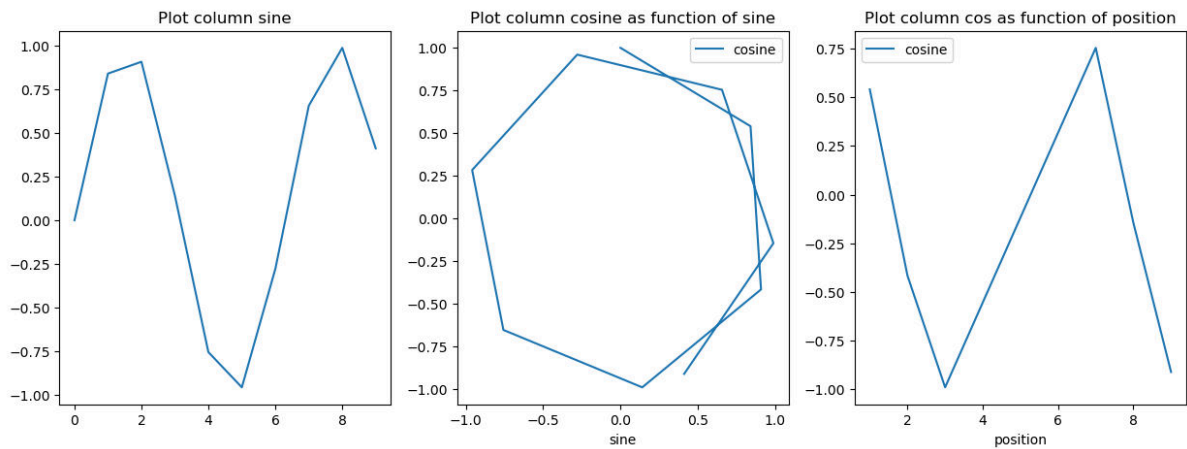
We mostly don't want to plot all columns at once

```
fig, ax=plt.subplots(1,3,figsize=(15,5))

df['sine'].plot(ax=ax[0]);
ax[0].set_title("Plot column sine");

df.plot(x='sine',y='cosine',ax=ax[1]);
ax[1].set_title("Plot column cosine as function of sine")

df[0<df['sine']].plot(x='position',y='cosine',ax=ax[2]);
ax[2].set_title("Plot column cos as function of position");
```

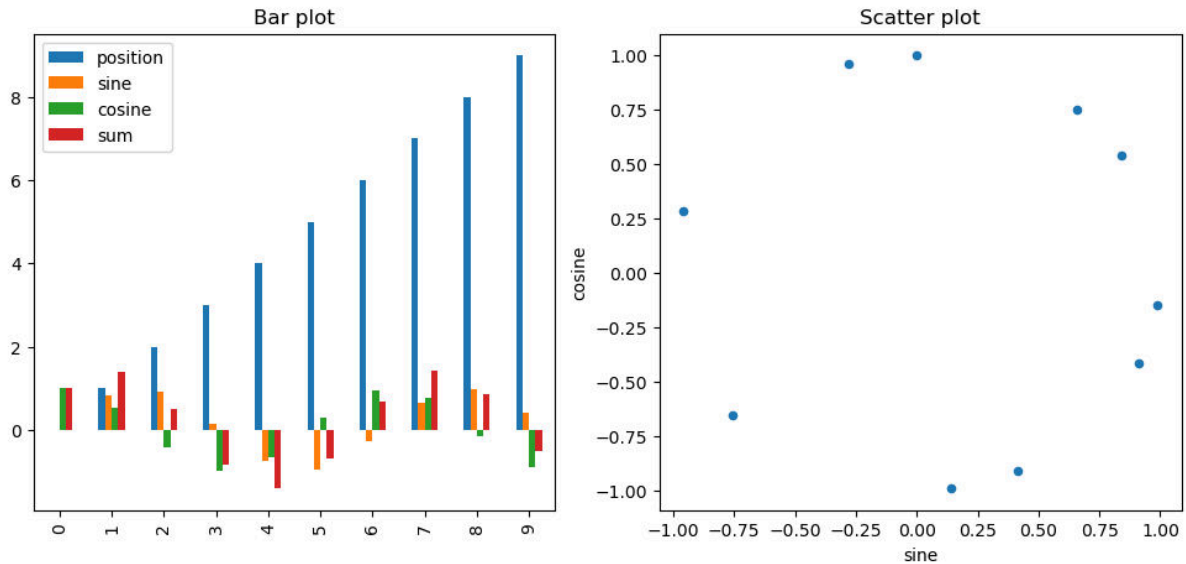


Different plotting styles

```
fig, (ax1,ax2)=plt.subplots(1,2,figsize=(12,5))

df.plot(kind='bar',ax=ax1);
ax1.set_title('Bar plot');

df.plot(kind='scatter',x='sine',y='cosine',ax=ax2);
ax2.set_title('Scatter plot');
```



Further plotting options can be found on [pandas visualization documentation](#)

0.8.8 Set operations with data frames

In our work we may produce several data frames that needs to be merged:

- Image features
- Meta data
- Sensor logs
- etc.

[Merging frames topic in Pandas documentation](#)

Concatenating frames

Add more rows with the same categories

Concatenation adds rows of two data frames with the same columns.

```
dfA = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']})

dfB = pd.DataFrame({'A': ['A4', 'A5', 'A6', 'A7'],
                    'B': ['B4', 'B5', 'B6', 'B7'],
                    'C': ['C4', 'C5', 'C6', 'C7'],
                    'D': ['D4', 'D5', 'D6', 'D7']})

frames = [dfA, dfB]
result = pd.concat(frames)
result
```

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
0	A4	B4	C4	D4
1	A5	B5	C5	D5
2	A6	B6	C6	D6
3	A7	B7	C7	D7

Concatenating this way has the disadvantage that the rows will be maintained from the original data frames. This can be avoided if you add an index vector when you concatenate.

```
dfA = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']},
                    index=[0, 1, 2, 3])

dfB = pd.DataFrame({'A': ['A4', 'A5', 'A6', 'A7'],
                    'B': ['B4', 'B5', 'B6', 'B7'],
                    'C': ['C4', 'C5', 'C6', 'C7'],
                    'D': ['D4', 'D5', 'D6', 'D7']},
                    index=[4, 5, 6, 7])

frames = [dfA, dfB]
result = pd.concat(frames)
result
```

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4
5	A5	B5	C5	D5
6	A6	B6	C6	D6
7	A7	B7	C7	D7

Merging data frames

Add columns with new categories, at least one column in common.

Merging is when you add columns to the data frame.

```
dfA = pd.DataFrame({'id' : [1,2,3,4],
                    'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']},
                    index=[0, 1, 2, 3])

dfB = pd.DataFrame({'id' : [1,2,3,4],
                    'X': ['B4', 'B5', 'B6', 'B7'],
```

(continues on next page)

(continued from previous page)

```
'Y': ['C4', 'C5', 'C6', 'C7'],  
'Z': ['D4', 'D5', 'D6', 'D7']},  
index=[1,2,3,4])  
  
result=pd.merge(dfA,dfB)  
result
```

	id	A	B	C	D	X	Y	Z
0	1	A0	B0	C0	D0	B4	C4	D4
1	2	A1	B1	C1	D1	B5	C5	D5
2	3	A2	B2	C2	D2	B6	C6	D6
3	4	A3	B3	C3	D3	B7	C7	D7

0.8.9 Create new data frame from selected columns

```
new = result[['A', 'C', 'D']]  
new.head()
```

	A	C	D
0	A0	C0	D0
1	A1	C1	D1
2	A2	C2	D2
3	A3	C3	D3

0.8.10 When are pandas data frames useful?

- Pandas is useful for **large data** - as long as it fits in the local memory.
- Really **big data** needs other options, e.g. [dask data frames](#)

How about big quantitative imaging?

0.9 Summary

- The importance of good data... what is good data
- Preparing data
- Famous data sets
- Augmentation
 - Transformations for increase the data
- Baseline algorithms
 - What is it?
 - How good is our baseline algorithm?
 - The confusion matrix
- Data frames

- Create, Select, Combine
- Plot

0.9.1 Next week

Noise and filters